
title: "Giải thích chi tiết mã nguồn OPC UA Communication Engine" project:
"OPCUACommDriver" stack: "WPF .NET 8, C#, OPC UA SDK 1.5.374, Serilog, BCrypt.Net,
Newtonsoft.Json, ASP.NET Core, Microsoft.Extensions.DependencyInjection" author:
"Documentation" date: "2026-06-19" language: "vi" version: "1.0.0"

Giải thích chi tiết mã nguồn: OPC UA Communication Engine

Tài liệu này được viết dành cho **lập trình viên junior** muốn hiểu sâu về cách một ứng dụng WPF .NET 8 thực sự được tổ chức và hoạt động. Mỗi dòng code, mỗi quyết định thiết kế đều được giải thích cặn kẽ bằng tiếng Việt kết hợp các thuật ngữ kỹ thuật tiếng Anh để bạn vừa hiểu vừa làm quen với ngôn ngữ chuyên ngành.

Mục lục

1. [Tổng quan kiến trúc](#)
2. [Enums - Các kiểu liệt kê](#)
3. [Models - Các lớp dữ liệu](#)
4. [Interfaces - Hợp đồng trừu tượng](#)
5. [Helpers - RelayCommand và Converters](#)
6. [Services/Auth - UserService và OperatorLockService](#)
7. [Services - ConfigurationService, DataCacheService, UiLogSink](#)
8. [Services/OpcUa - PlcConnection và PlcManager](#)
9. [Services/Protocols - Các giao thức khác](#)
10. [API Layer - ASP.NET Core nhúng trong WPF](#)
11. [ViewModels - MVVM Pattern](#)
12. [App.xaml.cs - Luồng khởi động](#)
13. [Views - Giao diện người dùng](#)
14. [Bugs và Fixes - Các lỗi thường gặp](#)
15. [Build Order Guide - Thứ tự xây dựng dự án](#)

Phần Phụ Lục - Kiến thức nền tảng (đọc khi đã "mất gốc")

- [Phụ lục A: C# nền tảng cho dự án này](#)

- [Phụ lục B: WPF và XAML nền tảng](#)
 - [Phụ lục C: MVVM - hiểu thật kỹ](#)
 - [Phụ lục D: OPC UA - khái niệm cốt lõi](#)
 - [Phụ lục E: Dependency Injection và vòng đời object](#)
 - [Phụ lục F: async/await và lập trình bất đồng bộ](#)
 - [Phụ lục G: Cấu trúc dự án và file .csproj](#)
 - [Phụ lục H: Dựng lại dự án từ con số 0 \(checklist\)](#)
 - [Phụ lục I: Bảng thuật ngữ \(Glossary\)](#)
-

Chương 1: Tổng quan kiến trúc

1.1. Dự án này làm gì?

OPC UA Communication Engine là một ứng dụng desktop (WPF) chạy trên Windows, có nhiệm vụ:

1. **Kết nối đến các thiết bị PLC** (Programmable Logic Controller) trong nhà máy công nghiệp thông qua nhiều giao thức khác nhau: OPC UA, Siemens S7, Mitsubishi MC Protocol, và Modbus TCP.
2. **Đọc và ghi giá trị các tag** — "tag" là tên gọi của một biến trong PLC (ví dụ: nhiệt độ lò, tốc độ động cơ, trạng thái van...).
3. **Cung cấp REST API và SignalR** để các ứng dụng khác (web, mobile) cũng có thể lấy dữ liệu từ PLC mà không cần cài phần mềm riêng.
4. **Quản lý người dùng** với phân quyền (Viewer, Operator, Admin) và JWT authentication.
5. **Hiển thị log real-time** trên giao diện để người vận hành theo dõi trạng thái hệ thống.

1.2. Tại sao dùng WPF thay vì ứng dụng Web?

WPF (Windows Presentation Foundation) được chọn vì:

- Ứng dụng công nghiệp thường chạy trên máy tính Windows đặt ngay tại nhà máy, **không cần kết nối internet liên tục**.
- WPF có thể chạy **liên tục 24/7** mà không phụ thuộc vào browser hay web server riêng.
- **Performance cao hơn** cho việc cập nhật dữ liệu real-time từ hàng trăm tag PLC.
- Có thể **nhúng ASP.NET Core** vào bên trong process WPF để đồng thời cung cấp API cho bên ngoài — đây là điểm đặc biệt nhất của project này.

1.3. Sơ đồ kiến trúc phân lớp (Layer Architecture)

Toàn bộ hệ thống được chia thành 6 lớp rõ ràng, mỗi lớp chỉ giao tiếp với lớp kế bên:



1.4. Giải thích chi tiết từng lớp

Lớp 1: WPF UI Layer — Giao diện người dùng

Đây là phần người dùng nhìn thấy và tương tác trực tiếp. Gồm các cửa sổ (Window) và hộp thoại (Dialog):

- **MainWindow** : Cửa sổ chính hiển thị bảng danh sách PLC, danh sách tag, log, và trạng thái kết nối. Đây là trung tâm điều hành của operator.
- **LoginWindow** : Màn hình đăng nhập khi khởi động ứng dụng. Yêu cầu username/password và xác thực qua **UserService**.
- **BrowseServerWindow** : Cửa sổ duyệt cây node (node tree) của OPC UA Server — giống như File Explorer nhưng dành cho cấu trúc dữ liệu PLC. Người dùng có thể chọn các node để thêm vào danh sách tag theo dõi.
- **Dialogs**: Các hộp thoại nhỏ như **AddPlcDialog** (thêm PLC mới), **EditPlcDialog** (chỉnh sửa cấu hình), **AddTagDialog** (thêm tag), **ChangePasswordDialog** (đổi mật khẩu), **UserManagementWindow** (quản lý tài khoản).

Nguyên tắc thiết kế UI Layer: View (giao diện) **không chứa logic nghiệp vụ**. View chỉ hiển thị dữ liệu từ ViewModel và gửi command lên ViewModel khi người dùng tương tác. Logic xử lý nằm ở ViewModel.

Lớp 2: ViewModel Layer — Logic hiển thị

ViewModel là "não" của giao diện, theo pattern **MVVM** (Model-View-ViewModel):

- **MainViewModel** : Quản lý toàn bộ state của **MainWindow** . Chứa danh sách PLC (**ObservableCollection<PlcDevice>**), danh sách log, thông tin user đang đăng nhập, các command (AddPlc, Connect, Disconnect, Save...).
- **BrowseServerViewModel** : Quản lý cây node trong **BrowseServerWindow** . Xử lý việc load nodes, lazy-expand, thêm tag.

Điểm mấu chốt: ViewModel **không biết gì về UI cụ thể** — không tham chiếu trực tiếp đến Button, TextBox, hay bất kỳ control nào. ViewModel chỉ expose properties (dữ liệu) và commands (hành động). WPF Data Binding tự động kết nối giữa ViewModel và UI.

```
Người dùng click Button "Kết nối"
      |
      v (Button.Command binding)
MainViewModel.ConnectSelectedCommand.Execute()
      |
      v
MainViewModel.ConnectSelectedAsync()
      |
      v (gọi service)
PlcManager.ConnectAsync(plcId)
```

Lớp 3: Service Layer — Nghiệp vụ

Chứa toàn bộ **business logic** (logic nghiệp vụ) không liên quan đến giao diện:

- **ConfigurationService** : Đọc/ghi file cấu hình JSON. Quản lý **AppConfiguration** chứa danh sách PLC và tag. Phát event **ConfigurationChanged** khi tải file mới.
- **UserService** : Toàn bộ logic quản lý tài khoản. Hash mật khẩu với BCrypt, validate credentials, quản lý active sessions, refresh token.
- **OperatorLockService** : Hệ thống khóa quyền điều khiển (operator lock). Đảm bảo chỉ một người vận hành được phép ghi giá trị vào PLC tại một thời điểm. Lưu trạng thái lock vào file JSON để chia sẻ giữa nhiều instance.
- **DataCacheService** : Cache giá trị tag để giảm load cho PLC và cung cấp truy cập nhanh cho API.
- **UiLogSink** : Custom Serilog sink — "cầu nối" giữa thư viện Serilog và UI. Khi có log mới, UiLogSink thêm vào **ObservableCollection<LogEntry>** trong MainViewModel để hiển thị lên màn hình.

Lớp 4: OPC UA / Protocol Layer — Giao tiếp thiết bị

Đây là lớp "nói chuyện" trực tiếp với PLC qua mạng:

- **PlcManager** : Quản lý tập hợp các kết nối PLC. Cung cấp API thống nhất để thêm/xóa/connect/disconnect PLC. Aggregate events từ tất cả connections.
- **PlcConnection** : Implement kết nối OPC UA (giao thức chuẩn quốc tế IEC 62541). Xử lý session management, subscription, auto-reconnect.
- **SiemensS7Connection** : Kết nối PLC Siemens S7 qua protocol độc quyền (dùng thư viện S7NetPlus).
- **MitsubishiMcConnection** : Kết nối PLC Mitsubishi qua MC Protocol.
- **ModbusTcpConnection** : Kết nối thiết bị Modbus TCP (dùng thư viện EasyModbus).

Tất cả 4 loại connection đều **implement cùng một interface** **IPlcConnection** — điều này cho phép **PlcManager** xử lý chúng một cách thống nhất mà không cần biết protocol cụ thể.

Lớp 5: API Layer — Giao tiếp bên ngoài

Một feature đặc biệt: project **nhúng một ASP.NET Core web server vào trong process WPF**:

- **ApiHostService** : Tạo và quản lý **IHost** (ASP.NET Core application) bên trong WPF. Khi MainWindow khởi động, API server cũng được start.
- **AuthController** : REST endpoint cho login, refresh token, logout.
- **PlcsController** : REST endpoint để lấy danh sách PLC, trạng thái kết nối.
- **TagsController** : REST endpoint để đọc/ghi giá trị tag.
- **LockController** : REST endpoint để acquire/release operator lock.
- **UsersController** : REST endpoint để quản lý tài khoản (chỉ Admin).
- **PlcHub** : SignalR Hub — client đăng ký và nhận push notification khi tag value thay đổi real-time qua WebSocket.

Lớp 6: Infrastructure — Tầng nền tảng

- **Serilog**: Thư viện logging với structured logging. Ghi log ra file, console, và UiLogSink.
- **DI Container** (**Microsoft.Extensions.DependencyInjection**): Quản lý vòng đời và dependency của tất cả objects. Services được register một lần, tự động inject vào constructors.
- **JSON Config Files**: Lưu cấu hình (**plc_config.json**), tài khoản người dùng (**users.json**), trạng thái lock (**lock_state.json**).

1.5. Luồng dữ liệu chính (Data Flow)

Để nắm bắt hệ thống, hãy theo dõi ba luồng dữ liệu quan trọng:

Luồng 1: Người dùng thêm PLC mới

```
[Người dùng click "Thêm PLC"]
|
v
MainViewModel.AddPlcCommand.Execute()
|
v
Hiện AddPlcDialog.ShowDialog()
|
v [Người dùng nhập địa chỉ IP, port, tên...]
dialog.Result = PlcDevice{...}
|
v
PlcDevices.Add(newPlc)          --> ObservableCollection --> UI tự cập nhật
ConfigService.MarkAsModified()  --> HasUnsavedChanges = true --> Save button enable
PlcManager.AddPlcAsync(newPlc)  --> tạo IPlcConnection object
|
v [Nếu ConnectAfterAdd = true]
Task.Run(() => PlcManager.ConnectAsync(newPlc.Id))
|
v
PlcConnection.ConnectAsync()    --> TCP connect --> OPC UA handshake --> Session
|
v
ConnectionStateChanged event fires
|
v
Dispatcher.InvokeAsync(() => plc.ConnectionState = Connected)
|
v
WPF Binding --> ListBoxItem đổi màu thành xanh lá
```

Luồng 2: OPC UA subscription nhận data mới

```
[PLC thay đổi giá trị sensor]
|
v
OPC UA server gửi publish notification (background thread)
|
v
PlcConnection.Session_Notification() callback
|
v
TagValueChanged event fires với (tagId, newValue, quality, timestamp)
|
v
DataCacheService.UpdateTag()      --> cập nhật in-memory cache
PlcHub.SendTagValueChanged()      --> push qua SignalR đến web clients
|
v [Dispatcher.BeginInvoke]
tag.UpdateValue(newValue, quality, timestamp)
| (kế thừa ObservableObject)
v
OnPropertyChanged("Value")
|
v
WPF Binding --> DataGridView cell tự cập nhật giá trị mới
```

Luồng 3: REST API client đọc tag

```
[Client: GET /api/tags/{plcId}/{tagId}]
|
v
TagsController.GetTagValue(plcId, tagId)
|
v
JWT middleware validate token
|
v
DataCacheService.GetTagValue() --> cache hit --> trả về ngay (fast path)
| nếu cache miss:
v
PlcManager.ReadTagAsync(plcId, tagId)
|
v
PlcConnection.ReadTagAsync() --> OPC UA ReadRequest --> PLC --> Response
|
v
return TagValueDto { Value, Quality, Timestamp }
|
v
JSON response 200 OK về client
```

Chương 2: Enums - Các kiểu liệt kê

2.1. Enum là gì và tại sao phải dùng?

Enum (enumeration — kiểu liệt kê) là một tập hợp các hằng số có tên. Thay vì dùng số nguyên `0`, `1`, `2` — những con số "vô hồn" không ai đoán được ý nghĩa — chúng ta dùng enum để code **tự mô tả** (self-documenting code).

Vấn đề khi không có enum:

```
// BAD CODE — số 2 nghĩa là gì? Ai hiểu được?  
if (user.Role == 2)  
{  
    ShowAdminPanel();  
}  
  
// BAD CODE — truyền số nguyên tùy tiện, compiler không kiểm tra được  
SetUserRole(userId, 99); // 99 không hợp lệ nhưng compiler không báo lỗi!
```

Với enum, code trở nên rõ ràng và an toàn:

```
// GOOD CODE — ai đọc cũng hiểu ngay  
if (user.Role == UserRole.Admin)  
{  
    ShowAdminPanel();  
}  
  
// GOOD CODE — compiler báo lỗi nếu truyền sai kiểu  
SetUserRole(userId, UserRole.Admin); // type-safe, compiler check at compile time
```

Lợi ích đầy đủ của enum:

1. **Type safety:** Compiler kiểm tra tại compile time — không thể truyền giá trị sai kiểu.
2. **IntelliSense support:** IDE tự gợi ý các giá trị hợp lệ khi gõ `.`.
3. **Refactoring an toàn:** Đổi tên enum value → IDE tự đổi toàn bộ references.
4. **Serialization linh hoạt:** JSON serializer có thể serialize thành string tên ("Admin") hoặc số (2) tùy cấu hình.
5. **Switch expression exhaustive:** Compiler cảnh báo khi bỏ sót case trong switch.

2.2. UserRole — Hệ thống phân quyền người dùng

```
public enum UserRole  
{  
    Viewer    = 0,    // Chỉ xem, không ghi  
    Operator  = 1,    // Vận hành — được ghi tag (cần operator lock)  
    Admin     = 2     // Toàn quyền — quản lý user, cấu hình  
}
```


Giải thích chi tiết từng giá trị:

Viewer = 0 — Người xem (quyền thấp nhất): - Đọc giá trị tag từ PLC (không ghi). - Xem trạng thái kết nối các PLC. - Xem log hệ thống. - **Không được**: ghi giá trị tag, thêm/sửa/xóa PLC, quản lý cấu hình, quản lý tài khoản. - Ví dụ use case: Kỹ sư giám sát chỉ theo dõi, không điều khiển.

Operator = 1 — Người vận hành: - Tất cả quyền của Viewer. - **Được phép ghi** giá trị tag (bật/tắt motor, thay đổi setpoint nhiệt độ, mở/đóng van...). - Nhưng phải **acquire operator lock** trước khi ghi — đảm bảo chỉ một người điều khiển tại một thời điểm. - **Không được**: thêm/xóa PLC, quản lý cấu hình hệ thống, quản lý tài khoản. - Ví dụ use case: Công nhân vận hành máy tại xưởng.

Admin = 2 — Quản trị viên (quyền cao nhất): - Tất cả quyền của Operator. - Thêm/sửa/xóa cấu hình PLC và tag. - Quản lý tài khoản người dùng (tạo, sửa, xóa, đặt lại mật khẩu). - Thay đổi API settings, Auth settings. - Acquire operator lock **không giới hạn thời gian** (unlimited lock). - Force-revoke lock của Operator khác. - Ví dụ use case: Kỹ sư hệ thống, trưởng ca.

Tại sao giá trị số tăng dần theo quyền hạn?

Thiết kế này cho phép **so sánh quyền bằng toán tử số học** — đơn giản và hiệu quả:

```
// Kiểm tra quyền: tất cả từ Operator trở lên mới được ghi tag
if ((int)user.Role >= (int)UserRole.Operator)
{
    await connection.WriteTagAsync(tagId, value);
}

// Helper methods rõ ràng
public static bool CanWriteTags(UserRole role) => role >= UserRole.Operator;
public static bool IsAdmin(UserRole role) => role == UserRole.Admin;
public static bool CanManageUsers(UserRole role) => role == UserRole.Admin;
```

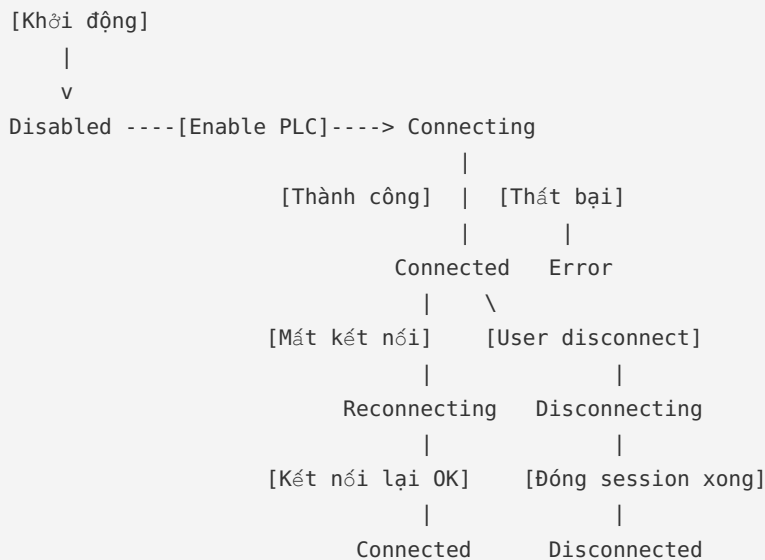
So sánh với cách không dùng số tăng dần:

```
// Phải liệt kê từng case – dễ thiếu sót
if (user.Role == UserRole.Operator || user.Role == UserRole.Admin)
{
    // Nếu sau này thêm UserRole.SuperAdmin = 3, phải sửa code này
}
```

2.3. PlcConnectionState — Vòng đời trạng thái kết nối PLC

```
public enum PlcConnectionState
{
    Disabled,          // Bị vô hiệu hóa chủ động trong config
    Connecting,        // Đang trong quá trình thiết lập kết nối
    Connected,         // Kết nối thành công, có thể đọc/ghi
    Disconnecting,     // Đang trong quá trình ngắt kết nối an toàn
    Disconnected,      // Đã ngắt kết nối hoàn toàn
    Reconnecting,      // Mất kết nối, đang tự động thử kết nối lại
    Error              // Lỗi nghiêm trọng, cần can thiệp thủ công
}
```

Đây là **state machine** (máy trạng thái) — mỗi trạng thái định nghĩa điều gì có thể xảy ra tiếp theo:



Giải thích tại sao cần từng trạng thái riêng biệt:

Disabled : PLC đang bị tắt **chủ động** bởi người dùng (flag `IsEnabled = false` trong cấu hình). Hệ thống **không thực hiện bất kỳ attempt kết nối** nào. Khác với **Disconnected** ở chỗ đây là **cố ý**, không phải do lỗi mạng. UI hiển thị màu xám, icon tắt.

Connecting : Đang thực hiện chuỗi bắt tay (handshake) để thiết lập kết nối: 1. Resolve DNS hostname → IP address. 2. Mở TCP socket đến port OPC UA (thường 4840). 3. Trao đổi certificates và thiết lập Secure Channel (TLS). 4. Tạo OPC UA Session (negotiation, activation). 5. Load và khôi phục subscriptions.

Cả chuỗi này có thể mất 2-15 giây. UI hiển thị spinner/loading indicator để người dùng biết đang xử lý.

Connected : Kết nối đang hoạt động bình thường. Có thể đọc/ghi tag, subscription nhận data. Đây là trạng thái **mong muốn**. UI hiển thị màu xanh lá, nút Disconnect enable, nút Connect disable.

Disconnecting : Người dùng chủ động ngắt kết nối và hệ thống đang thực hiện đóng kết nối đúng cách: 1. Hủy tất cả subscriptions đang active (gửi DeleteSubscriptions request đến server). 2. Đóng OPC UA Session gracefully (gửi CloseSession request). 3. Đóng TCP socket.

Quá trình này có thể mất 1-5 giây. Nếu không chờ mà force-close ngay, server sẽ giữ session "zombie" cho đến khi timeout (thường 30 giây) — lãng phí tài nguyên server và có thể gây conflict khi reconnect ngay sau đó.

Disconnected : Kết nối đã được ngắt **thành công và an toàn**. Không có error. Hệ thống **không tự động thử kết nối lại** từ trạng thái này — người dùng phải chủ động click "Kết nối". UI hiển thị màu xám.

Reconnecting : Kết nối bị đứt **ngoài ý muốn** (mạng mất đột ngột, PLC restart, switch mạng lỗi...) và hệ thống đang **tự động thử kết nối lại** với exponential backoff: - Lần 1: thử sau 5 giây. - Lần 2: thử sau 10 giây. - Lần 3: thử sau 20 giây. - Lần 4: thử sau 40 giây...

Khác với **Connecting** ở chỗ: - Đây là reconnect tự động, không phải connect lần đầu do user trigger. - Hệ thống cố gắng **khôi phục session** nếu còn trong grace period để giữ nguyên subscriptions (không phải đăng ký lại từ đầu). - UI hiển thị màu vàng, thông báo "Reconnecting... (attempt 3)".

Error : Lỗi **ng nghiêm trọng** không tự phục hồi được: - Certificate không hợp lệ hoặc bị từ chối. - Địa chỉ endpoint sai, không tìm thấy server. - Authentication thất bại (username/password OPC UA sai). - Phiên bản OPC UA không tương thích. - Đã thử reconnect tối đa số lần mà vẫn thất bại.

Người dùng cần **can thiệp thủ công**: kiểm tra lại IP, ping thiết bị, kiểm tra certificate, xem **LastError** để biết nguyên nhân cụ thể.

Tại sao UI cần phân biệt nhiều màu trạng thái?

Trong nhà máy, người vận hành nhìn màn hình từ xa qua camera hoặc từ góc khác. Màu sắc rõ ràng cho phép đánh giá tình trạng hệ thống chỉ trong một cái nhìn: - Xanh lá = Connected (OK, không cần chú ý) - Vàng = Connecting/Reconnecting (đang xử lý, chờ chút) - Đỏ = Error (cần xử lý ngay!) - Xám = Disabled/Disconnected (không hoạt động, bình thường nếu cố ý)

2.4. OpcUaSecurityPolicy — Thuật toán mã hóa OPC UA

```
public enum OpcUaSecurityPolicy
{
    None,                // Không mã hóa — chỉ dùng lab/test
    Basic128Rsa15,        // RSA 1024 + AES 128 — đã lỗi thời
    Basic256,             // RSA 1024 + AES 256 — deprecated
    Basic256Sha256,       // RSA 2048 + AES 256 + SHA-256 — phổ biến nhất
    Aes128Sha256RsaOaep,  // AES 128 + SHA-256 + RSA-OAEP — chuẩn mới
    Aes256Sha256RsaPss    // AES 256 + SHA-256 + RSA-PSS — mạnh nhất
}
```

OPC UA Security Policy định nghĩa **bộ thuật toán** (cipher suite) được dùng để bảo vệ kết nối. Để hiểu, cần biết sơ về 3 thuật toán:

- **RSA**: Thuật toán mã hóa bất đối xứng (public/private key). Mỗi bên có một cặp khóa: public key (chia sẻ công khai) và private key (giữ bí mật). Dùng để **trao đổi khóa bí mật** một cách an toàn qua kênh không tin cậy.
- **AES**: Thuật toán mã hóa đối xứng. Sau khi trao đổi khóa xong qua RSA, dùng AES để mã hóa **toàn bộ dữ liệu** — nhanh và hiệu quả hơn RSA nhiều lần.
- **SHA-256**: Hàm hash một chiều, dùng để **ký số** (digital signature) — xác minh tính toàn vẹn của dữ liệu.

Giải thích từng lựa chọn từ yếu đến mạnh:

None : Hoàn toàn không có bảo mật. Dữ liệu truyền dạng **plaintext** — ai sniff packet đều đọc được. Chỉ dùng trong: - Môi trường lab/development. - Mạng nội bộ cô lập hoàn toàn (không kết nối internet). - Đặc biệt: Siemens S7-1200/1500 dùng **None** ổn định nhất do tránh được vấn đề timeout khi renew Secure Channel.

Basic128Rsa15 : RSA 1024-bit (deprecated theo NIST) + AES 128-bit. Đây là policy **cũ nhất** trong OPC UA spec, chỉ tồn tại để tương thích với thiết bị legacy từ trước 2010. RSA 1024-bit không còn an toàn với máy tính hiện đại — có thể bị crack trong vài tuần với cluster tính toán lớn.

Basic256 : RSA 1024-bit + SHA-1 + AES 256-bit. Cũng **deprecated** vì SHA-1 bị chứng minh có collision vulnerability từ năm 2017. Chỉ dùng với thiết bị quá cũ không hỗ trợ policy mới hơn.

Basic256Sha256 : RSA 2048-bit + SHA-256 + AES 256-bit. Đây là **lựa chọn mặc định phổ biến nhất** hiện nay. Cân bằng tốt giữa bảo mật và khả năng tương thích. Hầu hết PLC hiện đại đều hỗ trợ policy này.

Aes128Sha256RsaOaep : AES 128-bit + SHA-256 + RSA-OAEP (Optimal Asymmetric Encryption Padding). OAEP là padding scheme hiện đại hơn PKCS#1 v1.5 (dùng trong Basic256Sha256) — giải quyết một số điểm yếu lý thuyết. Đây là **lựa chọn tốt** cho hệ thống mới triển khai.

Aes256Sha256RsaPss : AES 256-bit + SHA-256 + RSA-PSS (Probabilistic Signature Scheme).
Mức bảo mật cao nhất. RSA-PSS là chuẩn chữ ký số tiên tiến nhất hiện nay. Dùng cho môi trường yêu cầu compliance cao (FDA 21 CFR Part 11, ISO 27001, IEC 62443...).

Lưu ý thực tế quan trọng với Siemens S7-1200:

S7-1200 với **Basic256Sha256** mất **6-14 giây** để renew Secure Channel (xảy ra định kỳ mỗi **SecurityTokenLifetime**, thường 3600 giây). Điều này kích hoạt chuỗi sự kiện xấu:

1. S7-1200 bắt đầu renew Secure Channel → mất 10 giây
2. Trong 10 giây đó, KeepAlive không nhận được response
3. KeepAlive timeout (mặc định OPC UA SDK là 15 giây)
4. PlcConnection nghĩ session chết → trigger reconnect
5. Reconnect lại phải renew Secure Channel → vòng lặp vô tận

Giải pháp trong project: Tăng **OperationTimeout** lên 60,000ms (60 giây) và dùng **SecurityPolicy.None** cho S7-1200 trong mạng nội bộ an toàn.

2.5. OpcUaSecurityMode — Chế độ bảo mật áp dụng

```
public enum OpcUaSecurityMode
{
    None,           // Không ký, không mã hóa – plaintext
    Sign,           // Chỉ ký số – đảm bảo integrity
    SignAndEncrypt // Vừa ký số vừa mã hóa – bảo mật đầy đủ
}
```

Nếu **OpcUaSecurityPolicy** là **bộ công cụ** (RSA, AES, SHA...) thì **OpcUaSecurityMode** là **cách dùng công cụ đó**:

None : Dùng kết hợp với **SecurityPolicy.None**. Không có bảo mật gì cả — nhanh nhất, không overhead.

Sign : Chỉ **ký số** (digital signature) mà **không mã hóa** nội dung: - Ai cũng có thể đọc nội dung packet nếu sniff mạng (**không confidential**). - Nhưng không ai có thể **giả mạo hoặc sửa đổi** packet mà không bị phát hiện (**integrity guaranteed**). - Use case: Mạng nội bộ an toàn, cần xác thực tính toàn vẹn nhưng không lo nghe lén.

SignAndEncrypt : Vừa **ký số** vừa **mã hóa** — bảo mật đầy đủ: - **Confidentiality**: Không ai đọc được nội dung. - **Integrity**: Không ai sửa được nội dung mà không bị phát hiện. - **Authentication**: Xác minh đúng là server mình muốn kết nối (thông qua certificate). - Đây là chế độ **khuyến nghị** cho mọi hệ thống production.

Quan hệ giữa SecurityPolicy và SecurityMode:

SecurityPolicy = None + SecurityMode = None → Không bảo mật
SecurityPolicy = Basic256Sha256 + SecurityMode = Sign → Ký số, không mã hóa
SecurityPolicy = Basic256Sha256 + SecurityMode = SignAndEncrypt → Đầy đủ bảo mật

Không hợp lệ:

SecurityPolicy = None + SecurityMode = Sign → Lỗi: không có thuật toán để ký
SecurityPolicy = None + SecurityMode = SignAndEncrypt → Lỗi: không có thuật toán

2.6. TagQuality — Chất lượng giá trị tag

```
public enum TagQuality
{
    Unknown,    // Chưa biết – chưa đọc lần nào
    Good,       // Tốt – giá trị đáng tin cậy hoàn toàn
    Bad,        // Xấu – giá trị không đáng tin cậy
    Uncertain   // Không chắc – có thể đúng, có thể sai
}
```

Đây là khái niệm **đặc trưng và cực kỳ quan trọng** của giao thức OPC. Nguyên tắc cốt lõi:

Trong hệ thống công nghiệp, không chỉ giá trị đo được quan trọng — chất lượng của giá trị đó cũng quan trọng không kém.

Tình huống thực tế minh họa tầm quan trọng của TagQuality:

Cảm biến nhiệt độ lò hơi → trả về: 25°C

Nếu không có TagQuality:

Hệ thống thấy 25°C → nghĩ lò đang nguội → bật nhiệt tối đa

Nhưng thực ra cảm biến bị đứt dây → luôn trả về default 25°C

Lò có thể đang ở 800°C → bật thêm nhiệt → nguy hiểm!

Với TagQuality:

Giá trị: 25°C, Quality: Bad (cảm biến fault)

Hệ thống nhận ra Bad quality → KHÔNG bật nhiệt → kích hoạt alarm

Operator được cảnh báo → kiểm tra và sửa cảm biến

Giải thích từng giá trị:

Unknown : Trạng thái **ban đầu** của mọi tag khi khởi động. Ứng dụng chưa bao giờ đọc được giá trị từ PLC. UI hiển thị "-" hoặc "N/A" thay vì giá trị số.

Good : Giá trị được đọc thành công và đáng tin cậy: - Cảm biến hoạt động bình thường (không có error bit trong PLC). - Kết nối ổn định, không có timeout. - Giá trị trong range hợp lệ. - Đây là trạng thái **mong muốn** — hệ thống điều khiển có thể tin tưởng giá trị này.

Bad : Giá trị **không đáng tin cậy** — không nên dùng để ra quyết định: - Cảm biến hỏng: broken wire, sensor fault, out-of-range. - PLC báo lỗi cho tag này (error status bit set). - Kết nối đến PLC bị gián đoạn — giá trị cũ được giữ lại nhưng đánh dấu Bad. - Node không tồn tại hoặc quyền truy cập bị từ chối. - **Hệ thống PHẢI xử lý Bad quality**: dừng hành động, kích hoạt alarm, chuyển sang backup sensor.

Uncertain : Giá trị **có thể** đúng nhưng không đảm bảo 100%: - Kết nối vừa được phục hồi — giá trị là từ lần đọc gần nhất chưa có confirmation mới. - Cảm biến đang warm-up sau khi cắm điện (chưa đạt ổn định nhiệt). - PLC đang trong quá trình khởi động lại (cold start). - Dữ liệu từ field device qua wireless với signal yếu. - Hệ thống nên **xử lý thận trọng**: có thể dùng nhưng cần validation bổ sung.

Ảnh xạ từ OPC UA Status Code 32-bit:

StatusCode bits 30-31:

00 = Good	→ TagQuality.Good
01 = Uncertain	→ TagQuality.Uncertain
10 = Bad	→ TagQuality.Bad
11 = Bad	→ TagQuality.Bad (subtype)

Ví dụ status codes:

0x00000000 (Good)	→ Good
0x40920000 (UncertainSubNormal)	→ Uncertain
0x80350000 (BadNotConnected)	→ Bad
0x80340000 (BadDeviceFailure)	→ Bad
0xC0000000 (BadLastSampleNotAvailable)	→ Bad

2.7. TagDataType — Kiểu dữ liệu của tag

```
public enum TagDataType
{
    Unknown,          // Chưa xác định kiểu
    Boolean,          // true/false – 1 bit logic
    SByte,            // -128 đến 127 – số nguyên 8-bit có dấu
    Byte,             // 0 đến 255 – số nguyên 8-bit không dấu
    Int16,            // -32,768 đến 32,767
    UInt16,           // 0 đến 65,535
    Int32,            // -2,147,483,648 đến 2,147,483,647
    UInt32,           // 0 đến 4,294,967,295
    Int64,            //  $\pm 9.2 \times 10^{18}$  – số nguyên 64-bit có dấu
    UInt64,           // 0 đến  $1.8 \times 10^{19}$ 
    Float,            // IEEE 754 single precision – ~7 chữ số thập phân
    Double,           // IEEE 754 double precision – ~15 chữ số thập phân
    String,           // Chuỗi ký tự Unicode UTF-8
    DateTime,         // Ngày giờ UTC (DateTime.UtcNow)
    ByteString,       // Mảng byte thô (binary data, certificate...)
    XmlElement,       // Fragment XML (đặc thù OPC UA information model)
    NodeId,           // OPC UA Node Identifier
    Guid,             // Global Unique Identifier 128-bit
    Array             // Mảng của bất kỳ kiểu trên
}
```

Tại sao PLC dùng nhiều kiểu integer thay vì chỉ một kiểu?

PLC là thiết bị **nhúng** (embedded) với tài nguyên bộ nhớ giới hạn và thường phải chạy real-time với thời gian scan cycle vài milliseconds. **Mỗi byte đều quý giá:**

Ví dụ thực tế trong một PLC S7-1200:

- Motor_1_Start : Boolean (1 bit) – bật/tắt motor
- Motor_1_Speed : UInt16 (2 bytes) – tốc độ 0-3000 RPM
- Temperature_1 : Float (4 bytes) – nhiệt độ -100.0°C đến 9999.9°C
- TotalCycles : UInt32 (4 bytes) – bộ đếm 0-4,294,967,295
- ErrorMessage : String (varies) – mô tả lỗi

Nếu dùng Double cho tất cả:

- Motor_1_Start : Double (8 bytes) → lãng phí 7 bytes!
- Với 1000 variables $\times$ 7 bytes = 7,000 bytes lãng phí
- Trên PLC nhỏ có 10KB RAM → chiếm 70% bộ nhớ cho waste!

Ánh xạ TagDataType vào kiểu C#:


```
// Implementation trong PlcConnection khi nhận data từ OPC UA:
object ConvertOpcUaValue(DataValue dataValue, TagDataType targetType)
{
    return targetType switch
    {
        TagDataType.Boolean    => (bool)dataValue.Value,
        TagDataType.SByte      => (sbyte)dataValue.Value,
        TagDataType.Byte       => (byte)dataValue.Value,
        TagDataType.Int16      => (short)dataValue.Value,
        TagDataType.UInt16     => (ushort)dataValue.Value,
        TagDataType.Int32      => (int)dataValue.Value,
        TagDataType.UInt32     => (uint)dataValue.Value,
        TagDataType.Int64      => (long)dataValue.Value,
        TagDataType.UInt64     => (ulong)dataValue.Value,
        TagDataType.Float      => (float)dataValue.Value,
        TagDataType.Double     => (double)dataValue.Value,
        TagDataType.String     => (string)dataValue.Value,
        TagDataType.DateTime   => ((DateTime)dataValue.Value).ToUniversalTime(),
        TagDataType.ByteString => (byte[])dataValue.Value,
        _                      => dataValue.Value
    };
}
```

Tại sao **DateTime** phải **ToUniversalTime()**?

OPC UA spec yêu cầu timestamps phải là UTC. Nhưng một số PLC (đặc biệt Siemens S7-1200 firmware cũ) trả về local time. Gọi **ToUniversalTime()** đảm bảo consistency — tất cả timestamps trong hệ thống đều là UTC.

2.8. TagAccessMode — Chế độ truy cập tag

```
public enum TagAccessMode
{
    Read,          // Chỉ đọc – input từ cảm biến
    Write,         // Chỉ ghi – output đến actuator (hiếm)
    ReadWrite      // Đọc và ghi – setpoint, parameter
}
```

Read — Chỉ đọc: Tag chỉ nhận giá trị từ PLC, không ghi ngược lại. Điển hình: - Giá trị đo từ cảm biến vật lý (nhiệt độ PT100, áp suất 4-20mA, lưu lượng Coriolis...). - Bộ đếm xung (pulse counter) trong PLC. - Trạng thái đọc từ I/O card (Digital Input). - Ví dụ:

Temperature_PT100 = 85.3°C — chỉ đọc, không ai ghi vào nhiệt độ cảm biến.

Write — Chỉ ghi: Rất hiếm trong thực tế. Trường hợp đặc biệt khi output đến actuator và PLC không lưu lại giá trị hiện tại (write-only register).

ReadWrite — Đọc và ghi: Phổ biến nhất cho các biến điều khiển: - Setpoint nhiệt độ: đọc để hiển thị hiện tại, ghi để thay đổi. - Timer preset: đọc để biết thời gian đã đặt, ghi để

thay đổi. - Control flags: `Motor_Enable = true/false` — đọc trạng thái, ghi để bật/tắt. - PID parameters: Kp, Ki, Kd — đọc và điều chỉnh online.

Quan trọng — sự khác biệt giữa AccessMode trong app và permission trên PLC:

AccessMode trong app config: Viewer thấy tag này là ReadWrite
→ Có thể attempt ghi từ UI

PLC security level: Tag này yêu cầu authentication để ghi
→ PLC từ chối write nếu không có password OPC UA

Kết quả: WriteTagAsync() trả về false, OPC UA error "BadUserAccessDenied"

Bảo mật thực sự phải đến từ PLC (server side). AccessMode trong app chỉ là UI hint.

2.9. ProtocolType — Loại giao thức kết nối

```
public enum ProtocolType
{
    OpcUa,          // OPC Unified Architecture – chuẩn IEC 62541
    SiemensS7,      // Siemens S7 Communication Protocol (độc quyền)
    MitsubishiMc,   // Mitsubishi MC Protocol (MELSEC Communication)
    ModbusTcp       // Modbus TCP – chuẩn mở, phổ biến nhất thế giới
}
```

Mỗi protocol ra đời từ bối cảnh lịch sử khác nhau và có ưu nhược điểm riêng:

OpcUa — Open Platform Communications Unified Architecture: - Chuẩn **quốc tế** IEC 62541, do OPC Foundation phát triển (2006, cập nhật liên tục). - **Vendor-neutral**: Siemens, Rockwell, ABB, Mitsubishi, Omron... đều implement. - **Feature-rich**: Security built-in, Node browsing, Subscriptions, Events, Alarms, Historical Access. - **Phức tạp hơn** nhưng là tương lai của công nghiệp 4.0. - Port mặc định: 4840 (TCP) và 4843 (HTTPS/WSS).

SiemensS7 — S7Communication (S7Comm): - Giao thức **độc quyền của Siemens** — không có spec công khai (reverse-engineered). - Hỗ trợ tốt nhất với dòng SIMATIC S7 (200, 300, 400, 1200, 1500). - **Không cần cài OPC UA Server** trên PLC (S7-1500 có built-in OPC UA server từ firmware 2.0). - Nhanh, ổn định, nhưng **chỉ Siemens** — không dùng được với PLC hãng khác. - Thư viện .NET: S7NetPlus (open source, GitHub).

MitsubishiMc — MC Protocol (MELSEC Communication): - Giao thức của **Mitsubishi Electric** cho dòng MELSEC PLC. - Có tài liệu kỹ thuật công khai (khác S7Comm). - Phổ biến tại Nhật Bản, Hàn Quốc, và Đông Nam Á. - Port mặc định: 3000 (TCP) hoặc 5007 (UDP).

ModbusTcp — Modbus TCP/IP: - Chuẩn **mở** (open standard) từ năm 1979 (Modicon), được nâng cấp lên TCP năm 1999. - Hỗ trợ bởi **hàng nghìn thiết bị**: PLC, biến tần VFD, UPS, đồng hồ điện, cảm biến thông minh, RTU/remote I/O... - **Rất đơn giản**: 4 loại vùng nhớ,

8 function codes cơ bản. - **Không có security** built-in: plaintext, không authentication, không encryption. - Port mặc định: 502.

2.10. PlcType — Loại PLC cụ thể

```
public enum PlcType
{
    Generic,           // PLC thông dụng – dùng OPC UA chuẩn
    SiemensS7_1200,    // Siemens SIMATIC S7-1200 – entry-level compact PLC
    SiemensS7_1500,    // Siemens SIMATIC S7-1500 – high-performance modular
    SiemensS7_300,     // Siemens SIMATIC S7-300 – mid-range (thế hệ cũ)
    SiemensS7_400,     // Siemens SIMATIC S7-400 – high-end (thế hệ cũ)
    MitsubishiFX5U,    // Mitsubishi MELSEC iQ-F FX5U – compact modular
    MitsubishiQ,       // Mitsubishi MELSEC-Q – high-performance modular
    ModbusSlave        // Bất kỳ thiết bị hỗ trợ Modbus Slave
}
```

Tại sao cần **PlcType** riêng ngoài **ProtocolType** ?

ProtocolType nói "dùng giao thức nào", còn **PlcType** nói "model PLC cụ thể là gì". Cùng hãng nhưng mỗi model có đặc điểm kỹ thuật khác nhau ảnh hưởng đến cách đọc/ghi dữ liệu:

Siemens S7 — sự khác biệt giữa các model:

S7-300 / S7-400:

- Max PDU size: 240 bytes → đọc tối đa ~60 words mỗi request
- Data Block: DB1.DBB0 (byte), DB1.DBW2 (word), DB1.DBD4 (dword)
- Không hỗ trợ Optimized Data Block
- PUT/GET luôn available (không có security protection)

S7-1200:

- Max PDU size: 480 bytes → đọc tối đa ~120 words mỗi request
- Hỗ trợ Optimized DB (chỉ truy cập qua tên biến, không phải địa chỉ byte)
- Cần disable "PUT/GET communication" protection trong TIA Portal
- OPC UA Server built-in từ firmware 4.4
- Vấn đề: Renew Secure Channel mất 6-14 giây với Basic256Sha256

S7-1500:

- Max PDU size: 960 bytes → hiệu quả hơn nhiều với batch reads
- OPC UA Server built-in từ firmware 2.0 (tốt hơn S7-1200)
- Hỗ trợ TLS 1.3 và certificate management qua TIA Portal
- Web server built-in

Mitsubishi — sự khác biệt:

FX5U (iQ-F):

- Vùng nhớ: M (Marker/Coil), D (Data Register), Y (Output), X (Input)
- R (File Register), SM (Special Marker), SD (Special Register)
- Địa chỉ: D100, M200, Y0, X15
- Max read: 960 words / request

MELSEC-Q:

- Vùng nhớ phong phú hơn: SM, SD, X, Y, M, L, F, V, B, W, TC, TT, TS, CC, CT, CS, D, R, ZR
- Hỗ trợ nhiều CPU units trong một backplane
- Địa chỉ có thể dạng hex: M0001, W0100
- Hỗ trợ Multi-CPU configuration (đọc data từ các CPU khác nhau)

`PlcType` cho phép `ProtocolConnectionFactory` tạo đúng implementation với tham số tối ưu cho từng model.

Chương 3: Models - Các lớp dữ liệu

3.1. Tổng quan về Models trong MVVM

Trong kiến trúc MVVM, **Model** là tầng dữ liệu thuần túy. Models chứa: - Dữ liệu có thể serialize/deserialize (lưu file JSON, truyền qua mạng). - Computed properties (tính toán từ dữ liệu khác). - Validation logic đơn giản.

Models **không chứa**: - Logic nghiệp vụ phức tạp (đó là nhiệm vụ của Services). - Tham chiếu đến UI controls (Button, TextBox...). - Code truy cập mạng hoặc file trực tiếp.

Project có hai nhóm model:

Observable models — kế thừa `ObservableObject`: - UI tự động cập nhật khi data thay đổi thông qua WPF Data Binding. - Dùng cho: `PlcDevice`, `TagItem`, `MainViewModel`, `BrowseServerViewModel`. - Bất kỳ object nào cần binding trực tiếp lên UI đều phải kế thừa `ObservableObject`.

Plain data models — class C# thông thường (POCO): - Serialize/deserialize JSON để lưu file hay truyền qua API. - Dùng cho: `User`, `RefreshToken`, `ActiveSession`, `AppConfiguration`. - Không cần binding trực tiếp — UI refresh bằng cách load lại toàn bộ list.

3.2. ObservableObject.cs — Nền tảng của WPF Data Binding

Đây là **class quan trọng nhất** trong toàn bộ project. Tất cả ViewModel và các Model cần real-time UI update đều kế thừa từ class này.

```
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;

namespace OpcUaCommunicationEngine.Models;

public abstract class ObservableObject : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;
```

Phân tích từng dòng:

using System.ComponentModel;

Namespace chứa interface **INotifyPropertyChanged** — interface cốt lõi của WPF data binding. Khi một class implement interface này, WPF binding engine biết rằng class đó có khả năng thông báo khi property thay đổi.

using System.Runtime.CompilerServices;

Namespace chứa attribute **[CallerMemberName]** — "phép màu" của C# compiler cho phép tự động điền tên của property/method đang gọi vào parameter được đánh dấu. Giúp tránh hardcode string tên property và lỗi typo.

using System.Windows;

Namespace của WPF. Cần để truy cập **Application.Current** và **Dispatcher** — hai thứ thiết yếu để marshal UI updates về UI thread một cách an toàn từ background threads.

public abstract class ObservableObject

Keyword **abstract** cho biết đây là **lớp trừu tượng** — không thể tạo instance trực tiếp:

```
var obj = new ObservableObject(); // LỖI compile – cannot instantiate abstract class
var vm = new MainViewModel();      // OK – MainViewModel kế thừa ObservableObject
```

Abstract vì class này chỉ cung cấp cơ chế thông báo (notification mechanism), không cung cấp dữ liệu. Chỉ các subclass cụ thể với properties thực sự mới có ý nghĩa để observable.

: INotifyPropertyChanged

Dấu **:** trong C# có thể có hai nghĩa: - Kế thừa class: **class Dog : Animal** - Implement interface: **class Dog : IRunnable**

Ở đây là implement interface. **INotifyPropertyChanged** yêu cầu một điều duy nhất: phải có **event PropertyChangedEventHandler? PropertyChanged**.

Cơ chế hoạt động của WPF Data Binding với INotifyPropertyChanged:

```
<!-- XAML -->
<TextBox Text="{Binding DeviceName, Mode=TwoWay}"/>
```

Khi XAML binding được thiết lập, WPF engine thực hiện: 1. Lấy giá trị `DeviceName` lần đầu để hiển thị ban đầu. 2. **Đăng ký lắng nghe** event `PropertyChanged` của DataContext object. 3. Mỗi khi `PropertyChanged` fires với `propertyName == "DeviceName"`, WPF gọi getter `DeviceName` lại và cập nhật TextBox.

Nếu object không implement `INotifyPropertyChanged`, bước 2 không xảy ra → binding chỉ đọc một lần, không bao giờ cập nhật khi data thay đổi.

```
public event PropertyChangedEventHandler? PropertyChanged;
```

event : Keyword C# đặc biệt bảo vệ sự kiện. Khác với delegate thông thường: - Chỉ class chứa event mới có thể `Invoke()` (raise) nó. - Bên ngoài chỉ có thể `+=` (subscribe) hoặc `-` (unsubscribe). - Encapsulation: không ai ngoài `ObservableObject` được kích hoạt event tùy tiện.

PropertyChangedEventHandler? : Kiểu delegate chuẩn của .NET cho loại event này.

Signature: `void Handler(object? sender, PropertyChangedEventArgs e)`. Dấu `?` (nullable) — event có thể không có subscriber nào (giá trị null).

```
protected virtual void OnPropertyChanged([CallerMemberName] string? propertyName = null)
{
    var handler = PropertyChanged;
    if (handler == null) return;
```

```
protected virtual void OnPropertyChanged
```

Convention C#: method để raise event đặt tên `On` + tên event. `protected` để chỉ class này và subclass gọi được (UI code bên ngoài không gọi trực tiếp). `virtual` để subclass override thêm behavior (ví dụ: log mỗi khi property thay đổi).

```
[CallerMemberName] string? propertyName = null
```

Compiler attribute cho phép tự động điền tên caller:

```
// Không có [CallerMemberName] – phải hardcode string:
public string DeviceName
{
    get => _deviceName;
    set
    {
        _deviceName = value;
        OnPropertyChanged("DeviceName"); // Nếu gõ sai → "DevlceName" → binding không update!
    }
}

// Với [CallerMemberName] – compiler tự điền:
public string DeviceName
{
    get => _deviceName;
    set
    {
        _deviceName = value;
        OnPropertyChanged(); // "" → compiler tự điền "DeviceName" – không thể sai typo
    }
}
```

var handler = PropertyChanged;

Dòng này cực kỳ quan trọng về **thread safety**. Giải thích vấn đề:

```
// KHÔNG an toàn:
if (PropertyChanged != null)           // Thread A: check null → true
{
    PropertyChanged.Invoke(...);       // Thread B: unsubscribe → PropertyChanged = null
}                                       // Thread A: NullReferenceException!

// AN TOÀN – copy snapshot trước:
var handler = PropertyChanged;         // Thread A: copy reference
if (handler == null) return;           // Thread B: PropertyChanged = null (handler vẫn valid)
handler.Invoke(...);                   // Thread A: safe – handler còn reference đến delegate list cũ
```

Pattern này được gọi là **"copy-and-invoke"** — chuẩn thread-safe cho event invocation trong .NET.

if (handler == null) return;

Tối ưu hiệu năng đơn giản: nếu không có subscriber nào (phổ biến trong unit tests hay khi object mới tạo), không làm gì cả. Tránh tạo `PropertyChangedEventArgs` object vô ích.

```

        if (Application.Current?.Dispatcher != null &&
            !Application.Current.Dispatcher.CheckAccess())
        {
            Application.Current.Dispatcher.BeginInvoke(() =>
            {
                handler.Invoke(this, new PropertyChangedEventArgs(propertyName));
            });
        }
        else
        {
            handler.Invoke(this, new PropertyChangedEventArgs(propertyName));
        }
    }
}

```

Phần xử lý thread safety — trọng tâm của **ObservableObject**:

Quy tắc sắt của WPF: Chỉ **UI thread** (main thread / dispatcher thread) mới được cập nhật UI. Vi phạm quy tắc này gây ra exception:

```

System.InvalidOperationException:
"The calling thread cannot access this object
because a different thread owns it."

```

Nguồn gốc background thread trong project này:

OPC UA SDK dùng thread pool nội bộ để xử lý subscription notifications:

```

OPC UA SDK internal thread pool
→ PlcConnection.Session_Notification() callback
→ tagValueChanged event fires
→ MainViewModel.OnTagValueChanged()
→ tag.UpdateValue(newValue, ...)
→ tag.Value = newValue (property setter)
→ SetProperty(ref _value, newValue)
→ OnPropertyChanged() ← ĐANG Ở BACKGROUND THREAD!

```

Application.Current?.Dispatcher

- **Application.Current** : Singleton instance của WPF application. **?.** null-conditional vì nếu app đang shutdown, **Application.Current** có thể là null.
- **.Dispatcher** : Object quản lý message queue của UI thread. Mọi UI operation phải đi qua Dispatcher.

!Application.Current.Dispatcher.CheckAccess()

- **CheckAccess()** trả về **true** nếu thread hiện tại là UI thread.
- **!** đảo ngược: điều kiện **true** khi đang ở **background thread** (cần marshal).

Dispatcher.BeginInvoke(() => { ... })

- **BeginInvoke** : Gửi delegate vào message queue của UI thread — **asynchronous**.

- Background thread **không bị block** — tiếp tục làm việc ngay lập tức.
- UI thread sẽ xử lý delegate này khi rảnh (khoảng vài milliseconds sau).

Tại sao `BeginInvoke` (async) thay vì `Invoke` (sync)?

`Invoke` block background thread đến khi UI thread xử lý xong. Vấn đề: - Giảm throughput OPC UA: background thread phải chờ mỗi khi có tag update. - Nguy cơ deadlock trong một số tình huống phức tạp (UI thread đang chờ background thread, background thread đang chờ UI thread).

`BeginInvoke` không block → background thread tiếp tục đọc data từ PLC và nhận notifications tiếp theo → throughput cao hơn.

```
protected bool SetProperty<T>(ref T field, T value, [CallerMemberName] string? propertyName = null)
{
    if (EqualityComparer<T>.Default.Equals(field, value))
        return false;

    field = value;
    OnPropertyChanged(propertyName);
    return true;
}
```

`SetProperty<T>` — Giảm boilerplate code từ 7 dòng xuống 1 dòng:

Không có `SetProperty` :

```
private string _deviceName = "";
public string DeviceName
{
    get => _deviceName;
    set
    {
        if (_deviceName != value) // check equality
        {
            _deviceName = value; // set field
            OnPropertyChanged(); // notify
        }
    }
}
// 7 dòng mỗi property
```

Với `SetProperty` :

```
private string _deviceName = "";
public string DeviceName
{
    get => _deviceName;
    set => SetProperty(ref _deviceName, value); // 1 dòng
}
```

Với ViewModel thường có 10-20 observable properties, tiết kiệm 60-120 dòng code!

<T> — Generic type parameter:

Generic cho phép `SetProperty` hoạt động với **bất kỳ kiểu nào**: - `SetProperty<string>(ref _name, "New Name")` — string - `SetProperty<int>(ref _count, 42)` — integer - `SetProperty<bool>(ref _isConnected, true)` — boolean - `SetProperty<PlcConnectionState>(ref _state, PlcConnectionState.Connected)` — enum - `SetProperty<ObservableCollection<TagItem>>(ref _tags, newCollection)` — complex object

Không cần viết overload riêng cho từng kiểu.

ref T field

`ref` cho phép method thay đổi **biến của caller** (thay đổi backing field thực sự). Không có `ref`, method chỉ nhận copy và mọi thay đổi mất sau khi method return:

```
// ref: thay đổi backing field thực sự
private string _name = "";
SetProperty(ref _name, "Alice"); // _name = "Alice" sau khi return

// không ref:
void BadSetProperty<T>(T field, T value) { field = value; } // vô tác dụng!
```

EqualityComparer<T>.Default.Equals(field, value)

Tại sao không dùng `field == value` ?

Với generic type `T`, compiler không biết liệu `==` operator có được implement không.

`EqualityComparer<T>.Default` chọn cách so sánh phù hợp nhất: - Nếu `T` implement `IEquatable<T>` → dùng `IEquatable.Equals` (nhanh, chính xác). - Nếu không → dùng `object.Equals()` (fallback). - Với value types (int, bool, struct) → compare by value. - Với string → compare by content (`"abc".Equals("abc") = true` dù 2 object khác nhau).

return true/false

Trả về boolean cho phép caller thực hiện hành động bổ sung khi giá trị thực sự thay đổi:

```

public PlcConnectionState ConnectionState
{
    get => _connectionState;
    set
    {
        if (SetProperty(ref _connectionState, value))
        {
            // Chỉ chạy khi state THỰC SỰ thay đổi – tránh spam notifications
            OnPropertyChanged(nameof(ConnectionStateText)); // cập nhật computed property
            OnPropertyChanged(nameof(IsConnected));          // cập nhật derived bool property
            OnPropertyChanged(nameof(CanConnect));           // cập nhật CanExecute state
            Log.Information("State changed to {State}", value);
        }
    }
}
}

```

3.3. User.cs — Hệ thống model quản lý người dùng

File `User.cs` chứa 4 classes liên quan đến authentication và session management.

Class User — Tài khoản người dùng

```

public class User
{
    public string Id { get; set; } = Guid.NewGuid().ToString();
}

```

Id — Khóa định danh duy nhất:

`Guid.NewGuid()` tạo một UUID 128-bit ngẫu nhiên. Xác suất collision là 1 trong 2^{122} — về mặt thực tế là không thể. Ví dụ: `"3f2504e0-4f89-11d3-9a0c-0305e82c3301"`.

Tại sao GUID thay vì integer auto-increment?

- Dữ liệu lưu trong file JSON — không có database để generate auto-increment.
- GUID unique ngay khi tạo object, không cần roundtrip đến central server.
- Nếu sau này migrate sang database, GUID làm primary key không gây conflict.
- GUID không lộ thông tin về số lượng records (integer `userId=1` cho kẻ tấn công biết "có ít nhất 1 user").

`.ToString()` chuyển `Guid` struct thành `string` để lưu vào JSON dễ đọc.

```

public string Username { get; set; } = string.Empty;

```

Username — Tên đăng nhập: - Case-insensitive: "Admin", "admin", "ADMIN" đều match (validate bằng `OrdinalIgnoreCase`). - Phải unique trong hệ thống (UserService kiểm tra trước khi tạo). - `string.Empty` thay vì `null` hoặc `""` — convention C# hiện đại: rõ ràng về intent, tránh `NullReferenceException`.

```
public string PasswordHash { get; set; } = string.Empty;
```

PasswordHash — Hash mật khẩu, KHÔNG PHẢI mật khẩu gốc:

Nguyên tắc bảo mật cốt lõi: Không bao giờ lưu mật khẩu dạng: - Plaintext:

"mypassword123" → thảm họa nếu file bị đánh cắp. - Reversible encryption (AES, Base64):

"bXlwYXNzd29yZDEyMw==" → chỉ cần key để decode. - MD5/SHA1/SHA256 hash:

"d7f5bfa..." → rainbow table attack có thể crack trong vài phút.

Project dùng **BCrypt** — thuật toán hash đặc biệt cho mật khẩu:

BCrypt hash ví dụ:

\$2a\$11\$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy

Cấu trúc:

\$2a\$ → phiên bản BCrypt algorithm

\$11\$ → work factor: $2^{11} = 2048$ vòng hash

N9qo8uL0ickgx2ZMRZoMye → 22 ký tự salt (ngẫu nhiên, Base64)

IjZAgcfl7p92ldGxad68LJZdL17lhWy → 31 ký tự hash (Base64)

BCrypt khác gì SHA256?

1. **Salt tự động:** Mỗi lần hash có salt ngẫu nhiên riêng → cùng mật khẩu → 2 hash khác nhau. Chống rainbow table.
2. **Intentionally slow:** \$11\$ = 2048 vòng → ~0.15-0.3 giây mỗi hash. Đủ nhanh cho UX, quá chậm cho brute force (1 tỉ attempt = ~5000 năm).
3. **Adaptive:** Có thể tăng work factor lên 12, 13... khi CPU nhanh hơn để duy trì mức bảo mật.

```
public string DisplayName { get; set; } = string.Empty;  
public UserRole Role { get; set; } = UserRole.Viewer;  
public bool IsActive { get; set; } = true;  
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;  
public DateTime? LastLoginAt { get; set; }  
public int? LockDurationMinutes { get; set; }  
}
```

DisplayName : Tên thân thiện như "Nguyễn Văn An" — hiển thị trong UI và log. Khác với Username dùng để login.

Role = UserRole.Viewer : Default quyền thấp nhất. Nguyên tắc **Least Privilege** — không cấp dư quyền.

IsActive = true : Flag vô hiệu hóa tài khoản mà không xóa. Nhân viên nghỉ việc →

IsActive = false . Giữ lại audit trail. Có thể kích hoạt lại nếu cần.

`DateTime CreatedAt = DateTime.UtcNow` : **Luôn dùng UTC** cho timestamps lưu trữ. UTC không thay đổi theo múi giờ — đảm bảo consistency khi hệ thống chạy ở nhiều locations.

`DateTime? LastLoginAt` : Nullable vì user mới chưa từng đăng nhập → null. Theo dõi hoạt động: nếu user không đăng nhập trong 90 ngày → cảnh báo hoặc lock.

`int? LockDurationMinutes` : Nullable với ý nghĩa ba trạng thái: - `null` : Dùng giá trị mặc định từ `AuthSettings`. - `0` : Không giới hạn thời gian lock (admin phải mở thủ công). - `>0` : Lock trong N phút rồi tự mở.

Class `UserStorage` — Container serialize JSON

```
public class UserStorage
{
    public List<User> Users { get; set; } = new();
}
```

Tại sao cần wrapper class thay vì serialize `List<User>` trực tiếp?

Format JSON:

```
// Serialize List<User> trực tiếp: JSON array
[
  { "Id": "abc...", "Username": "admin" },
  { "Id": "xyz...", "Username": "op1" }
]

// Serialize UserStorage: JSON object
{
  "Users": [
    { "Id": "abc...", "Username": "admin" },
    { "Id": "xyz...", "Username": "op1" }
  ]
}
```

JSON object (`{}`) để mở rộng hơn:

```
{
  "Version": 2,
  "LastModified": "2026-06-19T10:30:00Z",
  "Checksum": "sha256:abcdef...",
  "Users": [...]
}
```

Thêm metadata mà không breaking change với file cũ.

`= new()` là C# 9 **target-typed new expression** — compiler suy ra `new List<User>()` từ type annotation. Khởi tạo empty list ngay, tránh `NullReferenceException` khi access trước khi load data.

Class RefreshToken — Token làm mới JWT

```
public class RefreshToken
{
    public string Token { get; set; } = string.Empty;
    public string UserId { get; set; } = string.Empty;
    public DateTime ExpiresAt { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public bool IsRevoked { get; set; } = false;
    public string? SessionId { get; set; }
}
```

JWT Authentication Flow — tại sao cần Refresh Token:

Scenario: User đăng nhập vào mobile app lúc 8:00 AM

8:00 AM: Login thành công

→ Server cấp: Access Token (hết hạn 15 phút) + Refresh Token (hết hạn 7 ngày)

8:05 AM: App gọi API đọc tag

→ Gửi Access Token trong header: "Authorization: Bearer <access_token>"

→ Server validate → OK

8:15 AM: Access Token hết hạn

→ App dùng Refresh Token để lấy Access Token mới

→ User không cần đăng nhập lại!

8:16 AM: Tiếp tục dùng Access Token mới

8:00 AM ngày thứ 8 (7 ngày sau): Refresh Token hết hạn

→ App phải yêu cầu user đăng nhập lại

Tại sao không dùng một Access Token thời hạn dài? Vì: - Nếu bị đánh cắp, attacker có thể dùng mãi mãi. - Không thể revoke JWT sau khi cấp (stateless). - Short-lived Access Token + long-lived Refresh Token = balance giữa security và UX.

Token — Chuỗi random 64 bytes → Base64 ~88 chars: - **Bí mật tuyệt đối** — nếu lộ, attacker impersonate user trong 7 ngày. - Server lưu **hash** của token (BCrypt hoặc SHA256), không phải plaintext. - Client giữ token gốc, gửi khi cần refresh.

IsRevoked — Thu hồi chủ động (trước khi hết hạn): - User logout → revoke ngay lập tức. - Admin force-logout → revoke. - Đổi mật khẩu → revoke tất cả refresh tokens của user này. - **Tại sao cần revoke?** JWT Access Token stateless không thể revoke, nhưng Refresh Token có state → có thể kiểm soát.

SessionId — Liên kết với Active Session: - Revoke Refresh Token → invalidate session tương ứng. - Invalidate session → revoke Refresh Token tương ứng. - Hai chiều để đảm bảo consistency.

Class ActiveSession — Theo dõi phiên đăng nhập

```
public class ActiveSession
{
    public string SessionId { get; set; } = Guid.NewGuid().ToString();
    public string UserId { get; set; } = string.Empty;
    public string Username { get; set; } = string.Empty;
    public string DeviceId { get; set; } = string.Empty;
    public string? DeviceName { get; set; }
    public DateTime LoginAt { get; set; } = DateTime.UtcNow;
    public DateTime LastActivityAt { get; set; } = DateTime.UtcNow;
    public bool IsActive { get; set; } = true;
    public DateTime? InvalidatedAt { get; set; }
    public string? InvalidReason { get; set; }
    public string? InvalidatedByDeviceName { get; set; }
}
```

Use case trong nhà máy:

Buổi sáng:

- Operator A login từ Workstation-01 → Session S1 (DeviceId: WS01, DeviceName: "Workstation-01")
- Kỹ sư B login từ Laptop → Session S2 (DeviceId: LAP-ENG-001, DeviceName: "Engineering Laptop")

12:00 PM: Operator A đi ăn trưa quên logout

2:00 PM: Admin phát hiện → force-close Session S1 qua UserManagement UI

Session S1:

IsActive = false
InvalidatedAt = 2:00 PM
InvalidReason = "AdminForced"
InvalidatedByDeviceName = "ADMIN-WORKSTATION-02" ← Ai đã đóng session

Khi Operator A quay lại:

- Mọi API call đều trả về 401 Unauthorized
- UI hiển thị "Session đã bị đóng, vui lòng đăng nhập lại"
- Operator A phải login lại

DeviceId — Định danh thiết bị: - Thường là hash của machine name + MAC address. - Enforce "chỉ 1 session active per device" — nếu login từ cùng device, session cũ bị invalidate.

LastActivityAt — Thời điểm hoạt động cuối: - Cập nhật mỗi khi API call được thực hiện. - Session timeout: nếu **LastActivityAt** > X giờ trước → auto-invalidate.

InvalidatedByDeviceName — Ai đã đóng session này:

Trong hệ thống yêu cầu **audit trail** (FDA 21 CFR Part 11 cho thiết bị y tế, ISO 13849 cho máy móc safety, IEC 62443 cho industrial cybersecurity), mọi hành động phải được ghi lại: ai làm gì, từ đâu, lúc nào. Field này phục vụ yêu cầu đó cho action "đóng session".

Chương 4: Interfaces - Hợp đồng trừu tượng

4.1. Interface là gì? Tại sao project này đặc biệt cần?

Interface là một "hợp đồng" (contract) định nghĩa **những gì** một class phải cung cấp mà không quan tâm **cách thực hiện**.

Ví dụ ổ cắm điện như Interface:

```
Interface IElectricalOutlet định nghĩa:
- ProvideVoltage(): int    (phải cung cấp điện áp)
- ProvideCurrent(): double (phải cung cấp cường độ)

VietnamOutlet : IElectricalOutlet
- ProvideVoltage() → 220V
- ProvideCurrent() → 16A

USOutlet : IElectricalOutlet
- ProvideVoltage() → 110V
- ProvideCurrent() → 15A
```

Thiết bị nào "biết về" `IElectricalOutlet` có thể dùng cả hai — không cần biết đây là ổ cắm Việt Nam hay Mỹ.

Vấn đề đặc thù của project này:

Project hỗ trợ 4 giao thức PLC với implementation **cực kỳ khác nhau**:

```
// OPC UA: endpoint URL, session, subscription
var session = await Session.Create(appConfig, endpoint, false, name, 60000, identity, null);
var result = await session.ReadAsync(null, 0, TimestampsToReturn.Both, nodesToRead, ct);

// Siemens S7: IP, rack, slot, PDU
var s7client = new Plc(CpuType.S71200, "192.168.1.10", rack: 0, slot: 1);
await s7client.OpenAsync();
var bytes = await s7client.ReadBytesAsync(DataType.DataBlock, db: 1, start: 0, count: 4);

// Modbus TCP: IP, port, unit ID, register address
var modbusClient = new ModbusClient("192.168.1.10", 502);
modbusClient.Connect();
int[] registers = modbusClient.ReadHoldingRegisters(startAddress: 0, quantity: 10);
```

Không có Interface → `PlcManager` phải biết tất cả chi tiết:


```
// ANTI-PATTERN: switch case khổng lồ
public async Task<TagValue?> ReadTagAsync(string plcId, string tagId)
{
    var device = GetDevice(plcId);
    switch (device.ProtocolType)
    {
        case ProtocolType.OpcUa:
            var opc = (OpcUaPlcConnection)GetConnection(plcId);
            return await opc.ReadOpcUaNodeAsync(tagId);
        case ProtocolType.SiemensS7:
            var s7 = (SiemensS7Connection)GetConnection(plcId);
            return await s7.ReadS7DataBlockAsync(tagId);
        // Thêm Profinet → phải sửa PlcManager!
        // Thêm DNP3 → phải sửa PlcManager!
        // Vi phạm Open/Closed Principle
    }
}
```

Với Interface → **PlcManager** không biết protocol cụ thể:

```
// GOOD: PlcManager chỉ nói chuyện qua interface
public async Task<TagValue?> ReadTagAsync(string plcId, string tagId, CancellationToken ct = default)
{
    var connection = _connections.GetValueOrDefault(plcId)
        ?? throw new KeyNotFoundException($"PLC '{plcId}' not found");
    return await connection.ReadTagAsync(tagId, ct); // polymorphism!
}
// Thêm Profinet → chỉ cần tạo ProfinetConnection : IPlcConnection
// PlcManager KHÔNG cần sửa
```

Lợi ích cụ thể trong project:

1. Unit Testing:

```
// Test MainViewModel mà không cần PLC thật
var mockManager = new MockPlcManager();
mockManager.AddPlc("plc1", new MockPlcConnection()
    .SetupRead("temperature", new TagValue(85.5, TagQuality.Good)));

var viewModel = new MainViewModel(mockManager, mockConfig, mockUserService, logger);
await viewModel.ConnectAsync("plc1");
Assert.Equal(85.5, viewModel.SelectedPlcTags.First().Value);
```

2. Open/Closed Principle: Thêm protocol mới (EtherNet/IP, Profinet, DNP3) → chỉ tạo class mới implement **IPlcConnection** và đăng ký trong factory → không sửa code cũ.

3. Dependency Injection:

```
// DI Container
services.AddSingleton<IPlcManager, PlcManager>(); // register
// MainViewModel nhận interface, không biết implementation
public MainViewModel(IPlcManager plcManager) { ... } // inject
```

4.2. IPlcConnection — Hợp đồng cho một kết nối PLC

IPlcConnection là **interface cốt lõi nhất** trong project. Mọi implementation (OpcUA, Siemens, Mitsubishi, Modbus) phải tuân thủ contract này.

4.2.1. Properties

```
public interface IPlcConnection
{
    PlcDevice Device { get; }
    PlcConnectionState ConnectionState { get; }
    bool IsConnected { get; }
    string? SessionId { get; }
    DateTime? LastConnectedTime { get; }
    DateTime? LastDisconnectedTime { get; }
    string? LastError { get; }
```

PlcDevice Device { get; }

Read-only (chỉ getter). Trả về config của PLC gắn với connection này. Bất biến sau khởi tạo — một connection object gắn cố định với một PLC.

PlcConnectionState ConnectionState { get; }

Read-only từ bên ngoài. Implementation nội bộ set trạng thái thông qua private setter. Tránh external code can thiệp sai vào state machine.

bool IsConnected { get; }

Shorthand property — thường là `ConnectionState == PlcConnectionState.Connected`. Cho phép viết code ngắn gọn hơn mà vẫn readable:

```
if (connection.IsConnected) { ... } // vs
if (connection.ConnectionState == PlcConnectionState.Connected) { ... }
```

string? SessionId { get; }

OPC UA session ID do server cấp. Hữu ích để trace request trong log — tìm tất cả log lines liên quan đến session cụ thể. Nullable vì chỉ có giá trị khi Connected.

DateTime? LastConnectedTime { get; } và DateTime? LastDisconnectedTime { get; }

Tracking lịch sử kết nối:

```
// Tính uptime
if (connection.IsConnected && connection.LastConnectedTime.HasValue)
{
    var uptime = DateTime.Now - connection.LastConnectedTime.Value;
    Console.WriteLine($"Connected for {uptime:hh\\:mm\\:ss}");
}
```

```
string? LastError { get; }
```

Mô tả lỗi chi tiết khi `ConnectionState == Error`. Hiện thị cho admin để debug: -
`"BadSecurityPolicyRejected (0x80791000): The security policy is not supported"` → đổi
 SecurityPolicy - `"Connection refused 192.168.1.50:4840"` → kiểm tra IP và port -
`"Certificate chain validation failed"` → import certificate

4.2.2. Events

```
event EventHandler<PlcConnectionState>? ConnectionStateChanged;
event EventHandler<TagValueChangedEventArgs>? TagValueChanged;
event EventHandler<PlcErrorEventArgs>? ErrorOccurred;
```

Tại sao dùng Event thay vì Polling?

```
POLLING – inefficient:
while (true) {
    var state = connection.ConnectionState; // check mỗi 100ms
    if (state != _lastState) { UpdateUI(state); }
    await Task.Delay(100);
}
// Tốn CPU, độ trễ tối đa 100ms

EVENT – efficient:
connection.ConnectionStateChanged += (s, newState) => UpdateUI(newState);
// Không tốn CPU khi không có thay đổi
// Phản ứng ngay lập tức (< 1ms sau khi state thay đổi)
```

```
event EventHandler<PlcConnectionState>? ConnectionStateChanged
```

Fired ngay khi trạng thái thay đổi. Subscribers: - `PlcManager`: Cập nhật `ConnectedCount` property và aggregate event lên. - `MainViewModel`: Cập nhật UI màu sắc PLC item trong list. - `ApiHostService`: Push SignalR notification ra web clients.

```
event EventHandler<TagValueChangedEventArgs>? TagValueChanged
```

Event quan trọng nhất về tần suất — có thể fire hàng trăm lần mỗi giây khi nhiều tag subscription cùng lúc. `TagValueChangedEventArgs` chứa đầy đủ thông tin:

```
public class TagValueChangedEventArgs : EventArgs
{
    public string PlcId { get; set; }
    public string TagId { get; set; }
    public object? OldValue { get; set; }
    public object? NewValue { get; set; }
    public TagQuality Quality { get; set; }
    public DateTime ClientTimestamp { get; set; }
    public DateTime? ServerTimestamp { get; set; }
}
```

event EventHandler<PlcErrorEventArgs>? ErrorOccurred

Fired cho mọi lỗi — không chỉ khi state chuyển sang **Error**. Ví dụ: read tag thất bại (nhưng connection vẫn OK), subscription notification có lỗi... Cung cấp chi tiết để log và debug.

4.2.3. Connection Management Methods

```
Task ConnectAsync(Cancellation_token cancellationToken = default);
Task DisconnectAsync(Cancellation_token cancellationToken = default);
Task ReconnectAsync(Cancellation_token cancellationToken = default);
```

Task ConnectAsync(Cancellation_token cancellationToken = default)

Task — bất đồng bộ (async). Tại sao bắt buộc phải async?

Quá trình kết nối OPC UA mất 2-15 giây:

DNS resolution:	10-500ms
TCP three-way handshake:	1-5ms
TLS Secure Channel:	50ms - 14,000ms (S7-1200 mất đến 14s)
OPC UA session create:	100-500ms
Session activate:	50-200ms
Load subscriptions:	100ms × N subscriptions

Synchronous method → UI thread block → cửa sổ đơ, không thể tương tác, không redraw.

Async method → UI thread tự do xử lý events, hiển thị progress indicator.

Cancellation_token cancellationToken = default

Cho phép hủy operation giữa chừng:

```
// Với timeout 30 giây:
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(30));
try
{
    await connection.ConnectAsync(cts.Token);
}
catch (OperationCanceledException)
{
    Logger.Warning("Connection to {Name} timed out after 30s", device.Name);
}
```

`= default` tương đương `= CancellationToken.None` — không thể cancel nếu caller không truyền token. Caller phải opt-in vào cancellation.

4.2.4. Tag Operation Methods

```
Task<TagValue?> ReadTagAsync(string tagId, CancellationToken cancellationToken = default);
Task<Dictionary<string, TagValue?>> ReadTagsAsync(IEnumerable<string> tagIds, CancellationToken cancellationToken = default);
Task<bool> WriteTagAsync(string tagId, object value, CancellationToken cancellationToken = default);
Task<Dictionary<string, bool>> WriteTagsAsync(Dictionary<string, object> tagValues, CancellationToken cancellationToken = default);
```

`Task<TagValue?> ReadTagAsync(string tagId, ...)`

Trả về nullable `TagValue?`: - `TagValue` với value, quality, timestamps khi thành công. - `null` khi: tag ID không tìm thấy trong config, kết nối mất, timeout.

```
// Caller phải handle null và Bad quality:
var result = await connection.ReadTagAsync("temperature_1");
if (result == null)
{
    Logger.Warning("ReadTagAsync returned null for 'temperature_1'");
    return defaultTemperature;
}
if (result.Quality == TagQuality.Bad)
{
    Logger.Warning("Temperature quality is Bad: {Value}", result.Value);
    TriggerAlarm("TEMPERATURE_SENSOR_FAULT");
    return;
}
ProcessTemperature((double)result.Value);
```

`Task<Dictionary<string, TagValue?>> ReadTagsAsync(IEnumerable<string> tagIds, ...)`

Batch read — đọc nhiều tag trong một request. Hiệu quả hơn nhiều lần `ReadTagAsync` tuần tự:

Sequential (10 tags × 5ms round-trip):

10 × 5ms = 50ms tổng cộng

Batch read (10 tags trong 1 request):

1 × 5ms + processing = ~8ms tổng cộng

Cải thiện: 6x nhanh hơn. Với 100 tags: 50ms vs 500ms.

OPC UA standard hỗ trợ đọc nhiều nodes trong một `ReadRequest`. Siemens S7 hỗ trợ Multi-Variable Read. Modbus hỗ trợ đọc range của registers.

`IEnumerable<string>` thay vì `List<string>`: method chỉ cần iterate một lần — không cần index, không cần modification. Caller có thể truyền bất kỳ sequence nào: `List<string>`, `string[]`, `HashSet<string>`, `IQueryable<string>` ...

object value trong `WriteTagAsync`:

```
// Caller truyền native C# type:
await connection.WriteTagAsync("temperature_setpoint", 85.5); // double
await connection.WriteTagAsync("motor_start", true);         // bool
await connection.WriteTagAsync("pump_speed", 1500);           // int
await connection.WriteTagAsync("device_name", "Pump-A01");    // string

// Implementation tự convert sang OPC UA DataValue:
var dataValue = new DataValue(ConvertToOpcUaVariant(tagId, value));
```

4.2.5. Subscription Management Methods

```
Task<bool> CreateSubscriptionAsync(string subscriptionName, int publishingInterval = 1000, CancellationT
Task<bool> RemoveSubscriptionAsync(string subscriptionName, CancellationTokens cancellationTokens = default
Task<bool> AddTagToSubscriptionAsync(string subscriptionName, string tagId, CancellationTokens cancellati
Task<bool> RemoveTagFromSubscriptionAsync(string subscriptionName, string tagId, CancellationTokens cancellati
```

OPC UA Subscription — tại sao không chỉ poll?

POLLING (ReadTagAsync mỗi giây):

```
App → ReadRequest{temperature} → PLC → Response{25.0°C} (1 giây)
App → ReadRequest{temperature} → PLC → Response{25.0°C} (không đổi)
App → ReadRequest{temperature} → PLC → Response{25.1°C} (đổi!)
```

Vấn đề: 2/3 request vô ích. Với 100 tags × 1 request/giây = 100 req/s tải PLC.

SUBSCRIPTION (server push):

```
App → CreateSubscription{publishingInterval: 1000ms}
App → AddMonitoredItem{temperature, samplingInterval: 500ms}
```

```
PLC → (sau 500ms nhiệt độ thay đổi) → Publish{temperature: 25.1°C, changed}
App nhận notification ngay khi có thay đổi, không cần poll!
```

Lợi ích:

- Latency thấp hơn: nhận ngay khi thay đổi, không đợi đến lần poll tiếp theo
- Ít traffic: chỉ gửi khi có thay đổi
- Ít tải PLC: PLC chủ động push, không phải trả lời 100 req/s

subscriptionName : Mỗi subscription có tên để quản lý theo nhóm: -

"DashboardSubscription" : Tất cả tags hiển thị trên dashboard. - **"AlarmSubscription"** : Chỉ alarm tags, **publishingInterval: 100ms** (phản ứng nhanh). - **"HistorianSubscription"** : Tags cần ghi vào historian, **publishingInterval: 5000ms** .

publishingInterval = 1000 (mặc định 1 giây):

Server gửi publish notification tối thiểu mỗi N milliseconds. Tag thay đổi giữa các interval vẫn được capture — OPC UA server buffer lại và gửi trong lần publish tiếp theo. Giảm xuống 100ms cho alarm monitoring.

4.2.6. Browse Methods — Đặc trưng OPC UA

```
Task<IEnumerable<OpcUaNode>> BrowseAsync(string? nodeId = null, CancellationToken cancellationToken = default)
Task<OpcUaNode?> GetNodeInfoAsync(string nodeId, CancellationToken cancellationToken = default);
```

BrowseAsync — Duyệt cây node của OPC UA Server:

OPC UA server có cấu trúc cây giống file system:

```

Objects (i=85)
└─ DeviceSet
  └─ PLC_Line_01 (namespace: ns=2, s=PLC_Line_01)
    └─ Temperature_Inlet [Float, Read, °C]
    └─ Temperature_Outlet [Float, Read, °C]
    └─ Pressure_Inlet [Float, Read, bar]
    └─ Pump_Control
      └─ Pump_Enable [Boolean, ReadWrite]
      └─ Pump_Speed [UInt16, ReadWrite, RPM]
      └─ Pump_Fault [Boolean, Read]
  └─ PLC_Line_02
  └─ ...

```

`BrowseAsync(nodeId)` trả về danh sách con trực tiếp của node có ID `nodeId`.

`BrowseAsync()` (không có nodeId) browse từ `Objects` folder (root).

Feature này **độc quyền của OPC UA** — Modbus và Siemens S7 native không có mechanism browse chuẩn (phải config tag address thủ công).

`GetNodeInfoAsync` — Metadata của một node:

```

var nodeInfo = await connection.GetNodeInfoAsync("ns=2;s=Temperature_Inlet");
// nodeInfo chứa:
// DisplayName: "Temperature Inlet"
// Description: "Inlet temperature of heat exchanger HX-001"
// DataType: Float
// AccessLevel: Read
// EngineeringUnit: "°C" (OPC UA EUInformation)
// EURange: {Low: -100.0, High: 500.0}

```

Thông tin này cực kỳ hữu ích cho `BrowseServerWindow` — người dùng thấy tên, kiểu, đơn vị đo và biết ngay tag này là gì mà không cần tra tài liệu.

4.3. IPlcManager — Hợp đồng quản lý nhiều PLC

`IPlcManager` quản lý **tập hợp** các `IPlcConnection`. Đây là **Facade Pattern** — giao diện đơn giản cho hệ thống phức tạp.

4.3.1. Aggregated Events

```

public interface IPlcManager
{
    event EventHandler<PlcConnectionState>? ConnectionStateChanged;
    event EventHandler<TagValueChangedEventArgs>? TagValueChanged;
    event EventHandler<PlcErrorEventArgs>? ErrorOccurred;
}

```

`IPlcManager` expose **cùng events** như `IPlcConnection` nhưng là aggregate từ tất cả connections:


```
MainViewModel đăng ký 1 lần: manager.TagValueChanged += Handler;
```

PlcManager nội bộ subscribe vào mọi connection:

```
connection1.TagValueChanged += (s, e) => TagValueChanged?.Invoke(this, e);  
connection2.TagValueChanged += (s, e) => TagValueChanged?.Invoke(this, e);  
connection3.TagValueChanged += (s, e) => TagValueChanged?.Invoke(this, e);
```

Khi PLC2 thay đổi data:

```
connection2.TagValueChanged fires  
→ PlcManager re-fires manager.TagValueChanged  
→ MainViewModel.Handler() nhận event (không quan tâm từ PLC nào)
```

Điều này đơn giản hóa MainViewModel đáng kể — không cần quản lý subscription lifecycle cho từng connection.

4.3.2. Properties

```
int PlcCount { get; }  
int ConnectedCount { get; }  
IReadOnlyDictionary<string, IPlcConnection> Connections { get; }
```

IReadOnlyDictionary<string, IPlcConnection>

Expose dictionary nhưng read-only:

```
// Caller có thể iterate và lookup:  
foreach (var (plcId, connection) in manager.Connections)  
{  
    var status = connection.ConnectionState;  
}  
  
// Nhưng KHÔNG thể modify:  
manager.Connections["newKey"] = someConnection; // Lỗi compile!  
manager.Connections.Remove("key"); // Lỗi compile!  
// Phải đi qua AddPlcAsync() và RemovePlcAsync() – ensure lifecycle control
```

4.3.3. CRUD và Initialization

```
Task InitializeAsync(CancellationToken cancellationToken = default);  
Task<IPlcConnection> AddPlcAsync(PlcDevice device, CancellationToken cancellationToken = default);  
Task<bool> RemovePlcAsync(string plcId, CancellationToken cancellationToken = default);  
IPlcConnection? GetConnection(string plcId);  
bool HasPlc(string plcId);
```

Task InitializeAsync : Gọi 1 lần khi startup: 1. Load danh sách PLC từ ConfigurationService. 2. Tạo **IPlcConnection** object cho mỗi PLC (dùng factory theo ProtocolType). 3. Subscribe events của từng connection. 4. Không connect — connect được trigger riêng sau khi UI ready.

IPlcConnection? GetConnection(string plcId) : Synchronous lookup (dictionary O(1)).
Nullable return → caller phải null-check.

bool HasPlc(string plcId) : Check trước khi add để validate uniqueness:

```
if (plcManager.HasPlc(newDevice.Id))
{
    throw new InvalidOperationException($"PLC '{newDevice.Id}' already exists");
}
await plcManager.AddPlcAsync(newDevice);
```

4.3.4. Bulk Operations

```
Task ConnectAsync(string plcId, CancellationToken cancellationToken = default);
Task DisconnectAsync(string plcId, CancellationToken cancellationToken = default);
Task ConnectAllAsync(CancellationToken cancellationToken = default);
Task DisconnectAllAsync(CancellationToken cancellationToken = default);
Task ReconnectAsync(string plcId, CancellationToken cancellationToken = default);
Task ReconnectAllAsync(CancellationToken cancellationToken = default);
```

ConnectAllAsync — parallel connection, key optimization:

```
// Implementation:
public async Task ConnectAllAsync(CancellationToken ct = default)
{
    var enabledConnections = _connections.Values
        .Where(c => c.Device.IsEnabled)
        .ToList();

    // Task.WhenAll: chạy tất cả connect đồng thời, chờ tất cả xong
    var tasks = enabledConnections.Select(c => c.ConnectAsync(ct));
    await Task.WhenAll(tasks);
}

// Kết quả timing:
// 5 PLC × mỗi cái 3 giây kết nối
// Sequential: 5 × 3s = 15 giây
// Parallel: max(3s, 3s, 3s, 3s, 3s) = 3 giây → 5x nhanh hơn!
```

4.3.5. Tag Operations và Status

```
Task<TagValue?> ReadTagAsync(string plcId, string tagId, CancellationToken cancellationToken = default);
Task<Dictionary<string, TagValue?>> ReadTagsAsync(string plcId, IEnumerable<string> tagIds, CancellationToken cancellationToken = default);
Task<bool> WriteTagAsync(string plcId, string tagId, object value, CancellationToken cancellationToken = default);
Task<Dictionary<string, bool>> WriteTagsAsync(string plcId, Dictionary<string, object> tagValues, CancellationToken cancellationToken = default);
Task<Dictionary<string, TagValue?>> ReadAllTagsAsync(string plcId, CancellationToken cancellationToken = default);
Task<IEnumerable<OpcUaNode>> BrowseAsync(string plcId, string? nodeId = null, CancellationToken cancellationToken = default);
Task<OpcUaNode?> GetNodeInfoAsync(string plcId, string nodeId, CancellationToken cancellationToken = default);
Dictionary<string, PlcStatus> GetAllStatus();
PlcStatus? GetStatus(string plcId);
```

Tất cả tag operations đều có thêm `string plcId` so với `IPlcConnection`. `IPlcManager` là **router**:

```
// PlcManager implementation – đơn giản và nhất quán
public async Task<TagValue?> ReadTagAsync(string plcId, string tagId, CancellationToken ct = default)
{
    var connection = _connections.GetValueOrDefault(plcId)
        ?? throw new KeyNotFoundException($"PLC '{plcId}' not found");
    return await connection.ReadTagAsync(tagId, ct);
}
```

`GetAllStatus()` và `GetStatus()` — synchronous snapshot:

```
// PlcStatus DTO:
public class PlcStatus
{
    public string PlcId { get; set; }
    public string PlcName { get; set; }
    public PlcConnectionState ConnectionState { get; set; }
    public bool IsConnected { get; set; }
    public int ConfiguredTagCount { get; set; }
    public DateTime? LastConnectedTime { get; set; }
    public DateTime? LastDisconnectedTime { get; set; }
    public string? LastError { get; set; }
    public string? SessionId { get; set; }
}
```

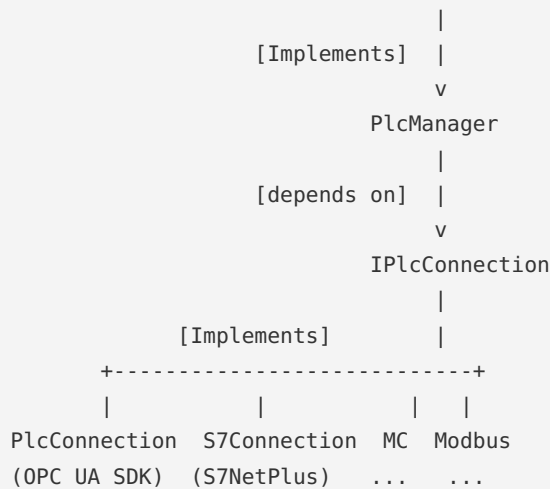
Dùng cho: - `GET /api/plcs` → `GetAllStatus()` → JSON array - `GET /api/plcs/{id}` → `GetStatus(id)` → JSON object - Dashboard summary: "3/5 PLCs online"

4.4. Kiến trúc phụ thuộc qua Interface

[Code cấp cao – không biết chi tiết implementation]

MainViewModel → depends on → IPlcManager

API Controllers → depends on → IPlcManager



Dependency arrows chỉ đi **xuống dưới**: - Code cấp cao (ViewModel, Controller) biết về interface, không biết implementation. - Implementation (PlcManager, PlcConnection...) không biết gì về ViewModel hay Controller. - Thay đổi implementation không ảnh hưởng code cấp cao. - Thay đổi interface → phải update tất cả implementation và caller.

Đây là hiện thực hóa đúng **SOLID principles**: - **S** (Single Responsibility): Mỗi class một trách nhiệm. - **O** (Open/Closed): Open for extension (thêm protocol mới), closed for modification (không sửa PlcManager). - **L** (Liskov Substitution): Bất kỳ **IPlcConnection** nào đều có thể thay thế nhau trong **PlcManager**. - **I** (Interface Segregation): **IPlcConnection** và **IPlcManager** tách biệt, không một interface "god object". - **D** (Dependency Inversion): Code cấp cao depend vào abstraction (interface), không depend vào concrete class.

[Phần tiếp theo — Chapters 5-15 — sẽ bao gồm: Helpers (RelayCommand, AsyncRelayCommand, Converters), Services/Auth, Services, OPC UA Layer chi tiết, Protocol implementations, API Layer (ASP.NET Core embedded), ViewModels, App.xaml.cs startup flow, Views/XAML patterns, Known Bugs và Fixes, và Build Order Guide]

Chương 5: Helpers - RelayCommand, AsyncRelayCommand, RelayCommand<T>

5.1 Tổng quan về ICommand Interface

Trong WPF (Windows Presentation Foundation), giao tiếp giữa UI và business logic được thực hiện thông qua pattern **MVVM (Model-View-ViewModel)**. Một trong những thành phần cốt lõi của MVVM chính là **ICommand interface**.

ICommand được định nghĩa trong namespace `System.Windows.Input` và chứa ba thành viên bắt buộc:

```
public interface ICommand
{
    event EventHandler? CanExecuteChanged;
    bool CanExecute(object? parameter);
    void Execute(object? parameter);
}
```

Tại sao WPF cần ICommand?

Trong WPF, các Button, MenuItem, và nhiều control khác có một property tên là `Command`. Khi user click button, WPF không gọi event handler trực tiếp - thay vào đó nó gọi `command.Execute()`. Điều này có những ưu điểm:

1. **Separation of Concerns:** Logic nằm trong ViewModel, không phải code-behind của View
2. **Enable/Disable tự động:** WPF gọi `CanExecute()` định kỳ để tự động enable/disable button
3. **Testability:** ViewModel (và command) có thể được unit test mà không cần UI

Thay vì implement ICommand cho từng action, **RelayCommand** đóng vai trò là một implementation tái sử dụng, nhận `Action` và `Func<bool>` qua constructor.

5.2 RelayCommand - Implementation chi tiết

```
using System.Windows.Input;

namespace OpcUaCommunicationEngine.Helpers;

public class RelayCommand : ICommand
{
    private readonly Action<object?> _execute;
    private readonly Func<object?, bool>? _canExecute;
```

Dòng 1: `using System.Windows.Input;` - import namespace chứa `ICommand` và `CommandManager`.

Dòng 5-6: Hai private readonly field: - `_execute` : Delegate kiểu `Action<object?>` - đây là logic sẽ chạy khi command được thực thi. `object?` vì `ICommand.Execute` nhận `object?`. - `_canExecute` : Delegate kiểu `Func<object?, bool>?` - nullable, nếu null thì command luôn enabled. Nhận parameter và trả về bool.

5.2.1 CanExecuteChanged và CommandManager.RequerySuggested

```
public event EventHandler? CanExecuteChanged
{
    add => CommandManager.RequerySuggested += value;
    remove => CommandManager.RequerySuggested -= value;
}
```

Đây là custom event accessor - thay vì tạo event riêng, chúng ta **delegate** (ủy thác) lên `CommandManager.RequerySuggested`.

CommandManager.RequerySuggested là gì?

`CommandManager` là static class của WPF, theo dõi trạng thái của UI. `RequerySuggested` là event mà WPF bắn ra khi nó "nghĩ ngờ" rằng `CanExecute` của các command có thể đã thay đổi - ví dụ khi: - User thay đổi focus - User nhập text - Một UI element thay đổi trạng thái

Khi event này bắn, WPF sẽ gọi lại `CanExecute()` trên tất cả các command đang được binding, và tự động enable/disable các control tương ứng.

Tại sao dùng add/remove accessor thay vì event thường?

Nếu chúng ta viết:

```
public event EventHandler? CanExecuteChanged;
```

thì WPF sẽ không biết khi nào cần refresh - chúng ta phải tự raise event này.

Bằng cách delegate lên `CommandManager.RequerySuggested`, chúng ta để WPF tự quản lý việc này một cách thông minh. Mỗi khi WPF gửi `RequerySuggested`, tất cả các button/control đang bind command này sẽ tự động cập nhật trạng thái.

5.2.2 Constructor và Null Guard

```
public RelayCommand(Action<object?> execute, Func<object?, bool>? canExecute = null)
{
    _execute = execute ?? throw new ArgumentNullException(nameof(execute));
    _canExecute = canExecute;
}
```

Dòng `_execute = execute ?? throw new ArgumentNullException(nameof(execute));`

Toán tử `??` (null-coalescing) ở đây được dùng như một **null guard**: - Nếu `execute` không null: gán vào `_execute` - Nếu `execute` null: throw `ArgumentNullException` với tên parameter

Đây là pattern bảo vệ an toàn - `RelayCommand` mà không có `execute` delegate thì hoàn toàn vô nghĩa. Fail nhanh tại constructor còn hơn là crash bí ẩn sau này khi `Execute()` được gọi với null delegate.

`nameof(execute)` trả về string `"execute"` - dùng cách này thay vì hard-code string để nếu rename parameter, compiler sẽ báo lỗi thay vì silently break.

```
public RelayCommand(Action execute, Func<bool>? canExecute = null)
    : this(_ => execute(), canExecute != null ? _ => canExecute() : null)
{
}
```

Constructor overload thứ hai - convenience constructor cho trường hợp command không cần parameter: - `_ => execute()` : lambda nhận parameter nhưng bỏ qua (dấu `_` là convention cho "discard") - `canExecute != null ? _ => canExecute() : null` : nếu có `canExecute`, wrap nó thành `Func<object?, bool>` ; nếu không thì null

`:this(...)` gọi constructor chính - tránh duplicate code.

5.2.3 CanExecute và Execute

```
public bool CanExecute(object? parameter) => _canExecute == null || _canExecute(parameter);
public void Execute(object? parameter) => _execute(parameter);
public void RaiseCanExecuteChanged() => CommandManager.InvalidateRequerySuggested();
```

CanExecute: Short-circuit evaluation: - Nếu `_canExecute == null` → true (không có điều kiện = luôn enabled) - Nếu có `_canExecute` → gọi delegate với parameter

Execute: Đơn giản gọi `_execute`. Không có null check vì đã guard trong constructor.

RaiseCanExecuteChanged: Gọi `CommandManager.InvalidateRequerySuggested()` - báo cho WPF biết cần re-evaluate `CanExecute` của tất cả commands. ViewModel gọi method này khi state thay đổi mà WPF không tự phát hiện được.

5.3 AsyncRelayCommand - Xử lý bất đồng bộ

```
public class AsyncRelayCommand : ICommand
{
    private readonly Func<Task> _execute;
    private readonly Func<bool>? _canExecute;
    private bool _isExecuting;
```

Sự khác biệt then chốt: `_execute` là `Func<Task>` thay vì `Action<object?>`. Command này dành cho các operation async như kết nối PLC, load dữ liệu từ file, v.v.

5.3.1 IsExecuting - Ngăn Re-entry

```
public bool IsExecuting
{
    get => _isExecuting;
    private set { _isExecuting = value; RaiseCanExecuteChanged(); }
}
```

Property `IsExecuting` có setter private - chỉ code trong class mới set được.

Tại sao cần IsExecuting?

Khi user click button kích hoạt một async operation (ví dụ kết nối OPC UA mất 3-5 giây), nếu không có guard, user có thể click liên tục và tạo ra nhiều connection attempts song song - gây race condition, resource leak, hoặc crash.

Cơ chế: 1. `IsExecuting = true` → setter gọi `RaiseCanExecuteChanged()` 2. WPF re-evaluate `CanExecute()` → trả về false vì `_isExecuting = true` 3. Button bị disable → user không click được nữa 4. Operation hoàn thành → `IsExecuting = false` → button enable lại

5.3.2 Tại sao Execute là `async void` chứ không phải `async Task` ?

```
public async void Execute(object? parameter)
{
    if (!CanExecute(parameter)) return;
    try { IsExecuting = true; await _execute(); }
    finally { IsExecuting = false; }
}
```


Đây là câu hỏi quan trọng về C# và WPF.

`ICommand.Execute` được định nghĩa trả về `void` :

```
void Execute(object? parameter);
```

Vì interface signature là `void` , chúng ta **buộc phải** implement với `void` . Không thể đổi thành `Task` vì vi phạm interface contract.

`async void` trong C# có những đặc điểm: - Exception không được propagate ra caller (caller không thể await) - Nếu có exception, nó sẽ bị throw vào SynchronizationContext (UI thread) - Thường được coi là anti-pattern, **ngoại trừ** event handlers và `ICommand.Execute`

Để bù đắp, `_execute` là `Func<Task>` - exception từ Task này sẽ propagate vào `async void Execute` , và nếu không được catch, sẽ crash ứng dụng (unhandled exception).

Khởi `try/finally` đảm bảo `IsExecuting = false` **luôn luôn** được gọi dù có exception hay không.

5.4 RelayCommand<T> - Generic Version

```
public class RelayCommand<T> : ICommand
{
    private readonly Action<T?> _execute;
    private readonly Func<T?, bool>? _canExecute;

    ...

    public bool CanExecute(object? parameter) => _canExecute == null || _canExecute((T?)parameter);
    public void Execute(object? parameter) => _execute((T?)parameter);
}
```

`RelayCommand<T>` cho phép type-safe commands. Thay vì nhận `object?` và phải cast trong body của action, casting được thực hiện tại boundary của command.

Casting `(T?)parameter` :

`ICommand.Execute` và `ICommand.CanExecute` nhận `object?` - đây là constraint từ interface. Khi WPF pass `CommandParameter` vào, nó là `object?` .

`(T?)parameter` là **unboxing/casting** từ `object?` sang `T?` . Nếu `T` là reference type thì là downcast. Nếu `T` là value type thì là unboxing. Nếu type không khớp, sẽ throw `InvalidCastException` .

Ví dụ sử dụng:

```
// ViewModel
ConnectCommand = new RelayCommand<PlcDevice>(device => ConnectToPlc(device));

// XAML
<Button Command="{Binding ConnectCommand}" CommandParameter="{Binding SelectedDevice}"/>
```

WPF sẽ pass `SelectedDevice` (kiểu `PlcDevice`) vào `Execute`, command cast về `PlcDevice?`, action nhận `PlcDevice?` typed - không cần manual cast trong body.

5.5 Tổng kết Helpers Pattern

Class	Execute Type	Use Case
<code>RelayCommand</code>	<code>Action<object?></code>	Sync operations, no parameter type
<code>RelayCommand<T></code>	<code>Action<T?></code>	Sync operations, typed parameter
<code>AsyncRelayCommand</code>	<code>Func<Task></code>	Async operations, prevents re-entry

Ba class này là backbone của toàn bộ command binding trong ứng dụng OPC UA, cho phép ViewModel expose các operation ra cho View mà không cần code-behind.

Chương 6: Services/Auth - UserService và OperatorLockService

6.1 UserService - Quản lý người dùng

6.1.1 Khởi tạo và Default Admin

Constructor của `UserService` thực hiện hai việc quan trọng:

- Load danh sách users từ JSON file:** Đọc file `users.json` và deserialize thành `List<UserAccount>`
- Tạo default admin nếu chưa có:** Nếu file không tồn tại hoặc chưa có user nào có role Admin, tạo user `admin/admin123`

```
// Pseudo-code của constructor
public UserService(string dataFilePath)
{
    _dataFilePath = dataFilePath;
    LoadUsersFromFile();

    if (!_users.Any(u => u.Role == UserRole.Admin))
    {
        CreateDefaultAdmin();
    }
}
```

Tại sao tạo default admin?

Đây là pattern phổ biến trong ứng dụng enterprise: lần đầu chạy, hệ thống cần ít nhất một account để vào cấu hình. Default `admin/admin123` là well-known credentials mà admin thực tế phải đổi ngay.

6.1.2 Thread Safety với lock(_lock)

```
private readonly object _lock = new();

public IEnumerable<UserAccount> GetAllUsers()
{
    lock (_lock)
    {
        return _users.ToList(); // ToList() để trả về copy, không expose internal list
    }
}
```

Tại sao cần lock trên mọi read operation?

Ứng dụng này có nhiều thread đồng thời: - **UI thread**: ViewModel gọi UserService để hiển thị danh sách users - **API thread (ASP.NET)**: HTTP request validate credentials - **Timer thread**: Auto-logout, session cleanup

Nếu không có lock, có thể xảy ra **race condition**: - Thread A đang đọc `_users` list - Thread B đang modify list (add/remove user) - Thread A đọc được list ở trạng thái corrupt

`lock(_lock)` đảm bảo chỉ một thread vào critical section tại một thời điểm. `_lock` là plain `object` - đây là pattern chuẩn cho locking.

`ToList()` tạo copy của internal list trước khi trả về - tránh caller giữ reference đến mutable internal collection.

6.1.3 BCrypt và Security Decisions

```
private string HashPassword(string password)
    => BCrypt.HashPassword(password, workFactor: 11);

private bool VerifyPassword(string password, string hash)
{
    try { return BCrypt.Verify(password, hash); }
    catch { return false; }
}
```

Tại sao BCrypt với workFactor=11?

BCrypt là password hashing algorithm được thiết kế đặc biệt để **chậm**. `workFactor=11` có nghĩa là $2^{11} = 2048$ iterations, mất khoảng 100-200ms trên CPU hiện đại.

So sánh với MD5/SHA256: - SHA256: < 1 microsecond per hash → attacker có thể thử hàng tỷ passwords/giây - BCrypt(11): ~150ms per hash → attacker chỉ thử được ~6 passwords/giây

Với database 1000 users bị leak, attacker cần thời gian **vô cùng lớn** để brute force.

Tại sao wrap trong try/catch?

`BCrypt.Verify` có thể throw exception nếu hash string bị corrupt (ví dụ truncated trong database). Thay vì để exception propagate lên, chúng ta return `false` - "password không hợp lệ". Đây là defensive programming: invalid hash = failed verification.

6.1.4 Username Normalization

```
public async Task<UserAccount> CreateUser(string username, ...)
{
    // Kiểm tra unique, case-insensitive
    if (_users.Any(u => string.Equals(u.Username, username, StringComparison.OrdinalIgnoreCase)))
        throw new InvalidOperationException("Username already exists");

    // Lưu lowercase
    var newUser = new UserAccount
    {
        Username = username.ToLowerInvariant(),
        ...
    };
}
```

OrdinalIgnoreCase: So sánh string theo byte value, không phụ thuộc culture, case-insensitive. Dùng cho username vì `"Admin"`, `"ADMIN"`, `"admin"` đều phải là cùng một user.

ToLowerInvariant(): Normalize về lowercase, dùng invariant culture (không phụ thuộc locale). Ví dụ: tiếng Thổ Nhĩ Kỳ có 'İ' uppercase → 'i' lowercase (khác ASCII 'i').

`ToLowerInvariant()` luôn dùng ASCII rules.

Lưu lowercase nhưng compare case-insensitive → đảm bảo consistency: dù user nhập `"Admin"` hay `"ADMIN"`, system luôn tìm thấy `"admin"` trong database.

6.1.5 UpdateUser - Partial Update Pattern

```
public UserAccount UpdateUser(string userId, string? newDisplayName, string? newEmail, UserRole? newRole)
{
    lock (_lock)
    {
        var user = GetUserById(userId) ?? throw new KeyNotFoundException();

        if (newDisplayName != null) user.DisplayName = newDisplayName;
        if (newEmail != null) user.Email = newEmail;
        if (newRole != null) user.Role = newRole.Value;

        SaveUsersToFile();
        return user;
    }
}
```

Partial update - chỉ cập nhật field nào được truyền vào (khác null). Điều này cho phép:

- Client chỉ gửi fields cần thay đổi - Fields không gửi giữ nguyên giá trị cũ

Nếu dùng full replace (ghi đè toàn bộ object), client phải gửi đầy đủ mọi field, dễ vô tình xóa dữ liệu.

6.1.6 DeleteUser - Ngăn Xóa Admin Cuối

```
public void DeleteUser(string userId)
{
    lock (_lock)
    {
        var user = GetUserById(userId) ?? throw new KeyNotFoundException();

        if (user.Role == UserRole.Admin)
        {
            var adminCount = _users.Count(u => u.Role == UserRole.Admin && u.IsActive);
            if (adminCount <= 1)
                throw new InvalidOperationException("Cannot delete the last active admin");
        }

        _users.Remove(user);
        SaveUsersToFile();
    }
}
```

Tại sao check last admin?

Nếu xóa admin cuối cùng, hệ thống sẽ không có cách nào để vào admin panel. Đây là **safety guard** - ngăn admin tự "lock mình ra ngoài".

Check `IsActive` vì admin bị disable cũng không login được - cần tính cả active admins.

6.2 OperatorLockService - Kiểm soát quyền Write

6.2.1 Mục đích và Design

Trong môi trường công nghiệp, nhiều operator có thể cùng monitor PLC nhưng **chỉ một người được phép write** tại một thời điểm. Viết đồng thời từ nhiều operator có thể gây:

- Race condition trên thiết bị
- Conflicting commands
- Safety hazards

`OperatorLockService` implement **pessimistic locking**: operator phải "claim" lock trước khi write. Lock có timeout để tự động expire nếu operator quên release.

State được persist vào `lock_state.json` để survive application restart.

6.2.2 TryAcquireLock - Logic chi tiết

```
public LockAcquireResult TryAcquireLock(
    string userId, string username, string displayName,
    UserRole role, int durationMinutes)
{
    lock (_lock)
    {
        // 1. Role check
        if (role < UserRole.Operator)
            return LockAcquireResult.Denied("Insufficient role");

        // 2. Same user có lock chưa expired → extend
        if (_currentLock?.UserId == userId && !IsLockExpired(_currentLock))
            return ExtendLock(durationMinutes);

        // 3. Khác user đang giữ lock → deny với holder info
        if (_currentLock != null && !IsLockExpired(_currentLock))
            return LockAcquireResult.Denied($"Held by {_currentLock.Username}");

        // 4. Tính expiry
        DateTime expiresAt;
        if (role >= UserRole.Admin || durationMinutes == 0)
            expiresAt = DateTime.MaxValue; // Unlimited
        else
            expiresAt = DateTime.UtcNow.AddMinutes(
                Math.Min(durationMinutes, MaxLockDurationMinutes));

        // 5. Tạo lock mới và atomic write
        _currentLock = new LockState { UserId = userId, ... ExpiresAt = expiresAt };
        AtomicWriteLockState();
        return LockAcquireResult.Success(_currentLock);
    }
}
```

Role check: `role < UserRole.Operator` - enum comparison. Viewer và Guest không được acquire lock.

Same user extend: Nếu operator đang giữ lock và request lại, thay vì deny, chúng ta extend thời gian. Điều này tránh operator bị interrupt giữa chừng vì lock expire.

DateTime.MaxValue cho unlimited: Admin không bị limit thời gian. `durationMinutes=0` cũng là tín hiệu "unlimited". `DateTime.MaxValue` là `9999-12-31` - thực tế là vĩnh viễn.

MaxLockDurationMinutes: Cap duration để ngăn operator giữ lock quá lâu. Ví dụ max 8 giờ - nếu operator quên release, system tự release sau 8 giờ.

6.2.3 Atomic File Write

```
private void AtomicWriteLockState()
{
    var tmpPath = _lockFilePath + ".tmp";
    var json = JsonConvert.SerializeObject(_currentLock, Formatting.Indented);

    File.WriteAllText(tmpPath, json);          // Ghi vào .tmp
    File.Move(tmpPath, _lockFilePath, true); // Rename atomic
}
```

Tại sao cần atomic write?

Nếu application crash giữa chừng khi đang ghi file: - File ghi một nửa → JSON corrupt -
Lần sau load → parse error → lock state mất

Giải pháp: Write to temp file (`lock_state.json.tmp`), sau đó rename. Rename là **atomic operation** trên hầu hết filesystems - file hoặc là file cũ hoặc là file mới, không có trạng thái trung gian.

`File.Move(tmpPath, _lockFilePath, true)` - parameter `true` là overwrite nếu destination đã tồn tại (C# 8.0+).

6.2.4 FileSystemWatcher - Multi-Instance Support

```
private void StartFileWatcher()
{
    _watcher = new FileSystemWatcher(Path.GetDirectoryName(_lockFilePath)!)
    {
        Filter = Path.GetFileName(_lockFilePath),
        NotifyFilter = NotifyFilters.LastWrite | NotifyFilters.FileName
    };

    _watcher.Changed += OnLockFileChanged;
    _watcher.EnableRaisingEvents = true;
}

private void OnLockFileChanged(object sender, FileSystemEventArgs e)
{
    if (_isInternalUpdate) return; // Bỏ qua self-triggered events

    Task.Delay(100).ContinueWith(_ => ReloadLockStateFromFile());
}
```

Tại sao cần FileSystemWatcher?

Trong môi trường production, có thể chạy nhiều instances của ứng dụng trên cùng máy (ví dụ: primary và backup). Nếu instance A acquire lock và ghi file, instance B cần biết về thay đổi này.

FileSystemWatcher monitor file system events và notify khi `lock_state.json` thay đổi từ bên ngoài.

`_isInternalUpdate` flag: Khi chính ứng dụng ghi file (atomic write), FileSystemWatcher cũng sẽ raise event `Changed`. Nếu không có guard, chúng ta sẽ reload file mà chính mình vừa ghi → vô hại nhưng lãng phí. `_isInternalUpdate = true` trước khi ghi, `= false` sau khi ghi xong.

100ms delay: Delay nhỏ trước khi reload - đảm bảo file đã được ghi xong hoàn toàn trước khi read. FileSystemWatcher event có thể fire ngay khi file bắt đầu được ghi, chưa có đủ dữ liệu.

6.2.5 StartTimeoutChecker - Background Timer

```
private void StartTimeoutChecker()
{
    _timeoutTimer = new System.Threading.Timer(
        callback: _ => CheckAndExpireLock(),
        state: null,
        dueTime: TimeSpan.FromSeconds(30),
        period: TimeSpan.FromSeconds(30));
}

private void CheckAndExpireLock()
{
    lock (_lock)
    {
        if (_currentLock != null && IsLockExpired(_currentLock))
        {
            var expiredLock = _currentLock;
            _currentLock = null;
            AtomicWriteLockState();
            LockExpired?.Invoke(this, expiredLock);
        }
    }
}
```

Timer chạy mỗi 30 giây, check nếu lock đã expired. Nếu expired: 1. Xóa lock khỏi memory 2. Ghi file (null lock state) 3. Raise event `LockExpired` để notify UI

30 giây là balance giữa responsiveness (lock expire nhanh) và performance (không check quá thường xuyên).

Chương 7: Services - ConfigurationService, DataCacheService, UiLogSink

7.1 UiLogSink - Đưa Log vào UI

7.1.1 Kiến trúc Logging

Ứng dụng dùng **Serilog** làm logging framework. Serilog có concept "sinks" - đầu ra của log. Mặc định có FileSink, ConsoleSink. **UiLogSink** là custom sink để hiển thị log trực tiếp trong UI.

```
public class UiLogSink : ILogEventSink
{
    private readonly ObservableCollection<LogEntry> _logEntries;
    private readonly System.Windows.Threading.Dispatcher _dispatcher;
    private readonly int _maxEntries;
```

ILogEventSink: Interface của Serilog, yêu cầu implement method **Emit(LogEvent)**.

ObservableCollection<LogEntry>: Collection đặc biệt trong WPF - tự động notify UI khi có item được add/remove. ListView/DataGrid binding lên collection này sẽ tự cập nhật.

Dispatcher: Object đại diện cho UI thread của WPF. Mọi thay đổi UI phải chạy trên UI thread.

7.1.2 Constructor - Capture Dispatcher

```
public UiLogSink(ObservableCollection<LogEntry> logEntries, int maxEntries = 1000)
{
    _logEntries = logEntries;
    _dispatcher = System.Windows.Application.Current.Dispatcher;
    _maxEntries = maxEntries;
}
```

Application.Current.Dispatcher - lấy Dispatcher của UI thread. **Phải** gọi trong constructor khi còn đang trên UI thread (vì **UiLogSink** được tạo trong App.xaml.cs trên UI thread).

Nếu không capture lúc này, sau này Emit() có thể được gọi từ background thread và không có cách nào lấy UI thread dispatcher.

_maxEntries = 1000 (default) - giới hạn số log entries để tránh memory leak. Log cứ accumulate mãi sẽ làm UI chậm và dùng nhiều RAM.

7.1.3 Emit - Thread-Safe UI Update

```
public void Emit(LogEvent logEvent)
{
    var entry = new LogEntry
    {
        Timestamp = logEvent.Timestamp.LocalDateTime,
        Level = GetLevelShortName(logEvent.Level),
        Message = logEvent.RenderMessage(),
        Exception = logEvent.Exception?.ToString()
    };

    _dispatcher.InvokeAsync(() =>
    {
        _logEntries.Add(entry);
        while (_logEntries.Count > _maxEntries)
            _logEntries.RemoveAt(0);
    });
}
```

Serilog gọi Emit() từ thread nào?

Emit() có thể được gọi từ bất kỳ thread nào đã log - background service, timer thread, API thread. Nhưng modify `_logEntries` (ObservableCollection) từ non-UI thread sẽ throw `InvalidOperationException`.

`_dispatcher.InvokeAsync()`: Marshal việc modify collection sang UI thread.

`InvokeAsync` (khác với `Invoke`) là fire-and-forget - Emit() return ngay mà không đợi UI thread process. Điều này tránh deadlock và blocking logger.

Trimming strategy:

```
while (_logEntries.Count > _maxEntries)
    _logEntries.RemoveAt(0);
```

`while` thay vì `if` - để handle trường hợp nhiều log entries được batch add. RemoveAt(0) xóa entry cũ nhất (FIFO - oldest out).

7.1.4 GetLevelShortName - Serilog Level Abbreviation

```
private static string GetLevelShortName(LogEventLevel level) => level switch
{
    LogEventLevel.Verbose => "VRB",
    LogEventLevel.Debug => "DBG",
    LogEventLevel.Information => "INF",
    LogEventLevel.Warning => "WRN",
    LogEventLevel.Error => "ERR",
    LogEventLevel.Fatal => "FTL",
    _ => "???"
};
```

Switch expression (C# 8.0) - concise thay vì if/else chain. Trả về 3-character abbreviation phù hợp với Serilog convention (`{Level:u3}`).

7.1.5 UiLogSinkExtensions - Fluent API

```
public static class UiLogSinkExtensions
{
    public static LoggerConfiguration UiSink(
        this LoggerSinkConfiguration sinkConfig,
        ObservableCollection<LogEntry> logEntries,
        int maxEntries = 1000)
        => sinkConfig.Sink(new UiLogSink(logEntries, maxEntries));
}
```

Extension method cho `LoggerSinkConfiguration` - cho phép dùng fluent syntax khi configure Serilog:

```
Log.Logger = new LoggerConfiguration()
    .WriteTo.File("app.log")
    .WriteTo.UiSink(LogEntries) // Extension method của chúng ta
    .CreateLogger();
```

`this LoggerSinkConfiguration sinkConfig` - đây là extension method, sinkConfig là object được extend.

7.2 ConfigurationService - Quản lý cấu hình ứng dụng

7.2.1 LoadConfigurationAsync

```
public async Task<AppConfiguration> LoadConfigurationAsync()
{
    if (!File.Exists(_configFilePath))
    {
        var defaultConfig = CreateDefaultConfiguration();
        await SaveConfigurationAsync(defaultConfig);
        return defaultConfig;
    }

    var json = await File.ReadAllTextAsync(_configFilePath);
    var config = JsonConvert.DeserializeObject<AppConfiguration>(json, GetJsonSettings());

    return config ?? CreateDefaultConfiguration();
}
```

Flow: 1. File không tồn tại → tạo default config, lưu lại, return 2. File tồn tại → đọc JSON → deserialize → return 3. JSON null (file rỗng hoặc "null") → fallback về default

`await File.ReadAllTextAsync` - async I/O, không block UI thread trong khi đọc file.

7.2.2 SaveConfigurationAsync và JSON Settings

```
private static JsonSerializerSettings GetJsonSettings() => new JsonSerializerSettings
{
    Formatting = Formatting.Indented,
    NullValueHandling = NullValueHandling.Ignore,
    DateFormatString = "yyyy-MM-dd HH:mm:ss"
};
```

Formatting.Indented: JSON output có indent (pretty print), dễ đọc khi mở file bằng text editor.

NullValueHandling.Ignore: Không serialize properties có giá trị null. Giúp JSON gọn hơn và tránh confusion khi null nghĩa là "not set" vs "set to null".

DateFormatString: Override default ISO 8601 format của Newtonsoft. `"yyyy-MM-dd HH:mm:ss"` là format thân thiện hơn cho non-technical users đọc config file. Tuy nhiên mất timezone info - cần cẩn thận nếu config được share cross-timezone.

7.2.3 ValidateConfiguration

```
public List<string> ValidateConfiguration(AppConfiguration config)
{
    var errors = new List<string>();

    // Check empty names
    if (config.PlcDevices.Any(d => string.IsNullOrEmpty(d.Name)))
        errors.Add("PLC device has empty name");

    // Check invalid URLs
    foreach (var device in config.PlcDevices)
    {
        if (!Uri.TryCreate(device.EndpointUrl, UriKind.Absolute, out _))
            errors.Add($"Invalid URL for device '{device.Name}': {device.EndpointUrl}");
    }

    // Check duplicate names
    var duplicates = config.PlcDevices
        .GroupBy(d => d.Name, StringComparer.OrdinalIgnoreCase)
        .Where(g => g.Count() > 1)
        .Select(g => g.Key);

    foreach (var dup in duplicates)
        errors.Add($"Duplicate device name: '{dup}'");

    return errors;
}
```

Validation return list of errors thay vì throw exception - cho phép collect **tất cả** errors và hiển thị một lần, thay vì fail on first error và user phải fix từng cái.

7.3 DataCacheService - In-Memory Cache

```
public class DataCacheService
{
    private readonly ConcurrentDictionary<string, TagValue> _cache = new();

    public void UpdateTagValue(string tagId, TagValue value)
        => _cache[tagId] = value;

    public TagValue? GetTagValue(string tagId)
        => _cache.TryGetValue(tagId, out var value) ? value : null;

    public IReadOnlyDictionary<string, TagValue> GetAllValues()
        => _cache;
}
```

ConcurrentDictionary: Thread-safe dictionary từ `System.Collections.Concurrent`. Cho phép multiple threads read/write đồng thời mà không cần manual lock.

Vai trò trong kiến trúc: - OPC UA Subscriptions update cache liên tục khi tag values thay đổi (ví dụ 100ms interval) - REST API endpoint serve latest values từ cache - UI binding cũng đọc từ cache

Nếu không có cache, mỗi API request phải query PLC trực tiếp → latency cao, tải nặng lên PLC network. Cache cho phép API respond trong microseconds.

Eventual consistency: Cache có thể lag 1 interval behind PLC real value. Với subscription interval 100ms, lag tối đa 100ms - chấp nhận được cho hầu hết use cases.

Chương 8: Services/OpcUa - PlcConnection và PlcManager

8.1 PlcConnection - Kết nối OPC UA

8.1.1 Key Fields và Ý Nghĩa

```
private readonly PlcDevice _device;
private readonly object _lock = new();
private Session? _session;
private ApplicationConfiguration? _appConfig;
private PlcConnectionState _connectionState = PlcConnectionState.Disconnected;
private CancellationTokenSource? _reconnectCts;
private bool _disposed;
private bool _isDisconnecting;
private bool _isReconnecting;
private SessionReconnectHandler? _reconnectHandler;
private DateTime _lastReconnectCompleteTime = DateTime.MinValue;
private const int KeepAliveSettlingPeriodMs = 2000;
private readonly ConcurrentDictionary<string, Subscription> _subscriptions = new();
private readonly ConcurrentDictionary<uint, (string TagId, string NodeId)> _monitoredItemMapping = new();
private static ILogger Logger => Log.Logger;
```

_device : Configuration của PLC này - endpoint URL, credentials, danh sách subscriptions.

_lock : Object dùng cho locking critical sections trong connect/disconnect flow.

_session : OPC UA Session object từ OPC Foundation SDK. Nullable vì chưa có session khi Disconnected.

_connectionState : Enum state machine - Disconnected, Connecting, Connected, Reconnecting, Error.

`_reconnectCts` : CancellationTokenSource để cancel reconnect loop khi disconnect được request.

`_disposed` : Flag cho IDisposable pattern.

`_isDisconnecting` : Guard để phân biệt intentional disconnect với connection loss.

`_isReconnecting` : Tránh start multiple reconnect attempts đồng thời.

`_reconnectHandler` : OPC UA SDK built-in handler cho Secure Channel reconnect.

`_lastReconnectCompleteTime` và `KeepAliveSettlingPeriodMs` : Sau khi reconnect xong, có thể có stale KeepAlive failure events trong queue. Settling period 2000ms ignore những events này.

`_subscriptions` : Dictionary mapping subscription group name → Subscription object.

`_monitoredItemMapping` : Dictionary mapping MonitoredItem handle (uint) → TagId và NodeId. Dùng để identify tag khi receive notification.

8.1.2 Logger là Property, không phải Field

```
// Property - đọc Log.Logger tại runtime
private static ILogger Logger => Log.Logger;

// vs Field - capture Log.Logger tại startup (WRONG pattern)
private static readonly ILogger _logger = Log.Logger;
```

Đây là một subtlety quan trọng của Serilog.

Serilog có static `Log.Logger` property. Khi app khởi động, `Log.Logger` ban đầu là `NullLogger` (không làm gì). Sau đó trong `App.xaml.cs`, chúng ta configure lại:

```
// App.xaml.cs OnStartup()
Log.Logger = new LoggerConfiguration()
    .WriteTo.File(...)
    .WriteTo.UiSink(LogEntries) // UI sink được add sau
    .CreateLogger();
```

Nếu `PlcConnection` capture `Log.Logger` vào field tại construction time (trước khi configure), field sẽ trở vào `NullLogger` cũ mãi mãi, kể cả sau khi `Log.Logger` được replace.

Với **property**, mỗi lần gọi `Logger.Information(...)` sẽ evaluate `=> Log.Logger` tại thời điểm đó, luôn trở vào logger hiện tại.

8.1.3 ConnectAsync - Flow chi tiết

Step 1: Guard checks

- └─ if (_disposed) throw ObjectDisposedException
- └─ if (already Connected/Connecting) return

Step 2: State transition

- └─ ConnectionState = Connecting

Step 3: Create ApplicationConfiguration

- └─ ApplicationName, URI, certificates
- └─ Transport quotas (message size, timeout)
- └─ Validate certs, create self-signed if needed

Step 4: Endpoint discovery with retry

- └─ SelectEndpointAsync(endpointUrl, useSecurity)
- └─ Retry 3 times với exponential backoff
- └─ Throw if all retries fail

Step 5: Session.Create()

- └─ Pass endpoint, clientCertificate, sessionTimeout=60s
- └─ Returns active OPC UA Session

Step 6: Wire events

- └─ _session.KeepAlive += OnKeepAlive
- └─ _session.Notification += OnNotification
- └─ _session.PublishError += OnPublishError

Step 7: State = Connected

- └─ ConnectionStateChanged event raised

Step 8: Create subscriptions

- └─ foreach SubscriptionGroup in _device.SubscriptionGroups where Enabled
 - └─ CreateSubscriptionAsync(group)

Bước 4 - Retry discovery: OPC UA discovery protocol là UDP-based và đôi khi unreliable trên một số networks. Retry 3 lần với delay giữa các lần giúp handle transient network issues.

Bước 6 - Wire events AFTER session created: Nếu wire events trước khi session tồn tại, event handlers có thể fire với null session → NullReferenceException.

8.1.4 DisconnectAsync - Flow chi tiết

```
Step 1: _isDisconnecting = true
        (prevents reconnect from triggering during cleanup)

Step 2: Dispose _reconnectHandler
        (stop SDK-level reconnect)

Step 3: StopAutoReconnect()
        (cancel our custom reconnect loop)

Step 4: Unwire events FIRST
├─ _session.KeepAlive -= OnKeepAlive
├─ _session.Notification -= OnNotification
└─ _session.PublishError -= OnPublishError

Step 5: Delete all subscriptions
        (graceful server-side cleanup)

Step 6: session.CloseAsync() with 5s timeout
        (tell server we're closing)

Step 7: session.Dispose()

Step 8: _session = null

Step 9: ConnectionState = Disconnected

Step 10: _isDisconnecting = false
```

Tại sao unwire events **TRƯỚC** khi close session?

Khi session đang được close, các KeepAlive failures hoặc network errors có thể trigger event handlers. Nếu event handler vẫn còn wired, chúng có thể try to reconnect trong khi chúng ta đang intentionally disconnecting → race condition.

5 second timeout cho CloseAsync: Nếu network đã drop, CloseAsync sẽ hang. Timeout đảm bảo disconnect không block indefinitely.

8.1.5 CRITICAL BUG: NodeId.Parse vs new NodeId

Đây là một trong những bug subtle nhất trong OPC UA programming với C# SDK.

```
// CORRECT - Numeric NodeId
var nodeId = NodeId.Parse("i=85");
// Result: NodeId { NamespaceIndex=0, IdType=Numeric, Identifier=85 }

// WRONG - String NodeId
var nodeId = new NodeId("i=85");
// Result: NodeId { NamespaceIndex=0, IdType=String, Identifier="i=85" }
```

Tại sao điều này quan trọng trong OPC UA protocol?

OPC UA NodeId có hai component: 1. **IdType**: Numeric, String, Guid, ByteString 2.

Identifier: Value tương ứng với type

"i=85" là **text representation** của một Numeric NodeId: - **i=** prefix có nghĩa là Numeric - **85** là identifier value

`NodeId.Parse("i=85")` **hiểu** text representation này và tạo Numeric NodeId với identifier=85.

`new NodeId("i=85")` **không parse** - nó tạo String NodeId với identifier là toàn bộ chuỗi "i=85".

Khi gửi request đến OPC UA server: - Server tìm node với Numeric identifier = 85 → FOUND (ví dụ: Objects folder) - Server tìm node với String identifier = "i=85" → NOT FOUND → BadNodeIdUnknown error

Bug này đặc biệt khó debug vì code compile và run không có error, chỉ receive BadNodeIdUnknown từ server - không biết tại sao.

Namespace index cũng quan trọng:

```
// NodeId với namespace 2
NodeId.Parse("ns=2;i=1001") // Correct
new NodeId("ns=2;i=1001")   // Wrong - String NodeId với identifier "ns=2;i=1001"
```

Luôn dùng `NodeId.Parse()` khi parse từ string representation.

8.1.6 KeepAlive Settling Period

```
private void OnKeepAlive(ISession session, KeepAliveEventArgs e)
{
    // Ignore KeepAlive events right after reconnect completes
    if ((DateTime.UtcNow - _lastReconnectCompleteTime).TotalMilliseconds < KeepAliveSettlingPeriodMs)
        return;

    if (e.Status.IsGood()) return; // Normal keepalive, all good

    if (_isDisconnecting) return; // Intentional disconnect, don't reconnect

    // KeepAlive failed → trigger reconnect
    StartAutoReconnect();
}
```

Tại sao cần 2000ms settling period?

OPC UA Session reconnect là hai-layer process: 1. **Layer 1**: SDK `SessionReconnectHandler` tự động reconnect Secure Channel 2. **Layer 2**: Sau khi Secure Channel up, application cần re-create Session

Khi Session reconnect hoàn thành và `_lastReconnectCompleteTime` được set, server vẫn có thể gửi KeepAlive responses cho **Secure Channel cũ** đang trong queue. Những responses này có thể có status là bad (vì channel đã close).

Nếu chúng ta process những events này ngay sau reconnect, chúng ta sẽ trigger reconnect lại một lần nữa → reconnect loop vô tận.

Settling period 2000ms = ignore events trong 2 giây sau reconnect → đủ thời gian cho queue cũ drain.

8.2 PlcManager - Quản lý nhiều PlcConnection

8.2.1 Kiến trúc

`PlcManager` là **facade** và **coordinator** cho nhiều `PlcConnection`. Application code không tương tác trực tiếp với `PlcConnection` - chỉ thông qua `PlcManager`.

```
public class PlcManager : IPlcManager, IDisposable
{
    private readonly ConcurrentDictionary<string, IPlcConnection> _connections = new();
    private readonly IPlcConnectionFactory _connectionFactory;
    private readonly IDataCacheService _dataCache;

    public event EventHandler<PlcConnectionStateChangedEventArgs>? ConnectionStateChanged;
    public event EventHandler<TagValueChangedEventArgs>? TagValueChanged;
}
```

ConcurrentDictionary<string, IPlcConnection>: Key là PlcId, value là connection. Thread-safe vì multiple threads có thể: - UI thread: disconnect một PLC - Timer thread: reconnect - API thread: đọc connection state

8.2.2 AddPlcAsync - Wire Events Pattern

```
public async Task AddPlcAsync(PlcDevice device)
{
    // 1. Factory creates connection
    var connection = _connectionFactory.Create(device);

    // 2. Wire events BEFORE adding to dictionary
    connection.ConnectionStateChanged += OnConnectionStateChanged;
    connection.TagValueChanged += OnTagValueChanged;

    // 3. Add to dictionary
    _connections[device.PlcId] = connection;

    // 4. Auto-connect if enabled
    if (device.AutoConnect)
        await connection.ConnectAsync();
}
```

Tại sao wire events trước khi add vào dictionary? Tránh race condition: nếu add trước rồi wire events sau, có thể có thread khác đọc từ dictionary và access connection chưa có event handlers.

8.2.3 RemovePlcAsync - Cleanup Order

```
public async Task RemovePlcAsync(string plcId)
{
    if (!_connections.TryRemove(plcId, out var connection))
        return;

    // CRITICAL: Unsubscribe events BEFORE dispose
    connection.ConnectionStateChanged -= OnConnectionStateChanged;
    connection.TagValueChanged -= OnTagValueChanged;

    await connection.DisconnectAsync();
    connection.Dispose();
}
```

Tại sao unsubscribe events **TRƯỚC** khi dispose?

Dispose sẽ trigger DisconnectAsync, có thể raise ConnectionStateChanged event. Nếu chúng ta đã remove connection khỏi dictionary nhưng chưa unsubscribe events, handler sẽ fire với một connection không còn tồn tại trong _connections → zombie event handler.

Zombie handler có thể: - Gây null reference nếu handler assumes connection còn trong dictionary - Update DataCache với stale data - Log confusing state changes

Thứ tự đúng: TryRemove khỏi dict → Unsubscribe events → Disconnect → Dispose.

8.2.4 ConnectAllAsync - Parallel Connection

```
public async Task ConnectAllAsync()
{
    var tasks = _connections.Values
        .Where(c => c.Device.Enabled && c.State != PlcConnectionState.Connected)
        .Select(c => c.ConnectAsync())
        .ToList();

    await Task.WhenAll(tasks);
}
```

Task.WhenAll(tasks) : Chạy tất cả connect operations **song song**. Nếu có 10 PLC, tổng thời gian là max(thời gian connect 1 PLC) thay vì sum.

So sánh: - Sequential: 10 PLCs x 3s = 30s startup time - Parallel (Task.WhenAll): max(3s) = 3s startup time

Điều này đặc biệt quan trọng khi startup - user không muốn đợi 30 giây trước khi UI responsive.

8.2.5 Two-Layer Reconnect Strategy

OPC UA reconnect có hai layer:

Layer 1 - SessionReconnectHandler (SDK):

```
Secure Channel drop detected
→ SessionReconnectHandler starts
→ Tries to restore Secure Channel
→ If successful: session.Reconnect() restores session state
→ SubscriptionTransferring event: re-transfer monitored items
```

Layer 2 - Custom reconnect loop:

```
KeepAlive failure detected (Layer 1 failed or timed out)
→ StartAutoReconnect() called
→ Loop với exponential backoff
→ ConnectAsync() creates brand new Session
→ OnConnected: re-create all subscriptions
```

Tại sao cần hai layer?

Layer 1 (SessionReconnectHandler) xử lý **transient failures** - brief network blip, server restart nhanh. SDK có thể restore session với subscriptions nguyên vẹn.

Layer 2 xử lý **prolonged failures** - server down lâu, Layer 1 timeout. Khi đó phải tạo session mới và re-subscribe.

Kết hợp hai layer cho phép fast recovery (Layer 1) cho common case, và reliable recovery (Layer 2) cho worse cases.

8.2.6 OnTagValueChanged - DataCache Update

```
private void OnTagValueChanged(object? sender, TagValueChangedEventArgs e)
{
    // Update cache
    _dataCache.UpdateTagValue(e.TagId, new TagValue
    {
        Value = e.Value,
        Quality = e.Quality,
        Timestamp = e.Timestamp
    });

    // Forward event to PlcManager subscribers
    TagValueChanged?.Invoke(this, e);
}
```

Flow khi tag thay đổi: 1. OPC UA server gửi Notification 2. SDK fires

`_session.Notification` event 3. `PlcConnection.OnNotification` processes notification, fires `TagValueChanged` 4. `PlcManager.OnTagValueChanged` receives event 5. Update DataCache (để REST API serve latest value) 6. Forward event (ViewModel có thể subscribe để update UI)

DataCache và UI update xảy ra tại cùng một thời điểm từ cùng một event. DataCache update là thread-safe (ConcurrentDictionary). UI update phải marshal sang UI thread qua Dispatcher.

8.3 Tổng kết kiến trúc OPC UA Layer

```
PlcManager
├─ PlcConnection (PLC-A)
│   └─ OPC UA Session
│       ├── Subscription (Group-1)
│       │   └─ MonitoredItem (Tag-1)
│       │       └─ MonitoredItem (Tag-2)
│       └─ Subscription (Group-2)
│           └─ MonitoredItem (Tag-3)
└─ Events: StateChanged, TagValueChanged
├─ PlcConnection (PLC-B)
└─ ...

DataCacheService
└─ ConcurrentDictionary<tagId, TagValue>
    (updated by PlcManager.OnTagValueChanged)

REST API
└─ GET /api/tags/{id} → reads from DataCacheService
```

Kiến trúc phân lớp rõ ràng: - **PlcConnection** : biết về OPC UA protocol - **PlcManager** : biết về multiple PLCs, routing events - **DataCacheService** : biết về caching - REST API: biết về HTTP - ViewModel: biết về UI

Mỗi layer chỉ communicate với layer liền kề - loose coupling, dễ test và maintain.

PHẦN PHỤ LỤC - KIẾN THỨC NỀN TẢNG

Phần này được viết riêng cho trường hợp bạn đã lâu không lập trình và cảm thấy "mất gốc". Nó **không thay thế** các chương ở trên mà **bổ sung** phần kiến thức nền — để khi đọc lại từng dòng code ở Chương 1–15 bạn không bị hổng. Hãy đọc phụ lục này **trước**, rồi quay lại các chương chi tiết.

Phụ lục A: C# nền tảng cho dự án này

Bạn không cần biết toàn bộ C#. Bạn chỉ cần nắm chắc những thứ project này thực sự dùng. Dưới đây là đúng những thứ đó.

A.1. Namespace, class, và file

```
namespace OpcUaCommunicationEngine.Models; // "địa chỉ" logic của code

public class PlcDevice // class = bản thiết kế (blueprint) cho object
{
    // nội dung class ở đây
}
```

- **namespace**: nhóm các class lại theo chức năng, tránh trùng tên.
`OpcUaCommunicationEngine.Models` nghĩa là class này thuộc nhóm "Models" của ứng dụng.
- **class**: khuôn mẫu. Từ một class bạn tạo ra nhiều **object** (thể hiện cụ thể) bằng `new PlcDevice()`.
- Dấu `;` sau namespace (thay vì `{ }`) là cú pháp **file-scoped namespace** của C# 10+ — cả file thuộc namespace đó, đỡ phải thụt lề.

A.2. Field vs Property — phân biệt CỰC KỲ quan trọng

Đây là chỗ người mới hay lẫn. Project này dùng cả hai khắp nơi.

```
public class Example
{
    private int _count; // FIELD: biến lưu trữ thật sự, thường private, đặt tên _camelCase

    public int Count // PROPERTY: "cổng" có kiểm soát để đọc/ghi field
    {
        get { return _count; } // chạy khi ai đó ĐỌC Count
        set { _count = value; } // chạy khi ai đó GHI Count (value = giá trị được gán)
    }

    public string Name { get; set; } // AUTO-PROPERTY: compiler tự tạo field ẩn bên trong

    public string Url => $"http://{Name}"; // EXPRESSION-BODIED property: chỉ đọc, tính toán mỗi lần gọi
}
```

Tại sao quan trọng với project này? Trong file `PlcConnection.cs` có dòng:

```
private static ILogger Logger => Log.Logger; // PROPERTY (có =>), KHÔNG phải field
```

Nếu viết thành field `private static ILogger Logger = Log.Logger;` thì giá trị được "chụp" **một lần duy nhất** lúc khởi tạo. Nhưng `App.xaml.cs` thay `Log.Logger` mới **sau khi đăng nhập** (để thêm UI sink). Vì đây là **property** `=> Log.Logger`, mỗi lần dùng `Logger` nó đọc lại `Log.Logger` mới nhất → log mới hiện được lên UI. Đây là một quyết định thiết kế tinh tế, và nó dựa hoàn toàn vào việc hiểu field khác property.

A.3. Access modifiers (mức truy cập)

Từ khóa	Ai dùng được
<code>public</code>	Mọi nơi
<code>private</code>	Chỉ trong cùng class
<code>protected</code>	Class này và class con (kế thừa)
<code>internal</code>	Trong cùng project (assembly)
<code>private static</code>	Chỉ trong class, và dùng chung cho mọi object (không gắn với 1 instance)

`static` nghĩa là "thuộc về class chứ không thuộc về object cụ thể". `Log.Logger` là static vì cả app chỉ có một logger.

A.4. Constructor (hàm khởi tạo)

```
public class UserService
{
    private readonly string _usersFilePath;

    public UserService(AuthSettings settings) // constructor: trùng tên class, không có kiểu trả về
    {
        _usersFilePath = settings.UsersFilePath; // chạy ngay khi new UserService(...)
        _storage = LoadOrCreateStorage();
    }
}
```

- Constructor chạy **một lần** khi object được tạo bằng `new`.
- `readonly` = gán một lần trong constructor rồi không đổi được nữa → an toàn hơn.
- Việc nhận `AuthSettings settings` qua constructor chính là **Dependency Injection** (xem Phụ lục E).

A.5. Interface và abstract class

```
public interface IPlcConnection // INTERFACE: chỉ là "hợp đồng", không có code thân hàm
{
    bool IsConnected { get; }
    Task<bool> ConnectAsync(CancellationTokens ct = default);
}

public class PlcConnection : IPlcConnection // class này CAM KẾT thực hiện đủ hợp đồng
{
    public bool IsConnected { get; private set; }
    public async Task<bool> ConnectAsync(CancellationTokens ct = default) { /* code thật */ }
}
```

- **Interface** = danh sách những thứ một class PHẢI có, nhưng không nói LÀM thế nào. Tên thường bắt đầu bằng **I**.
- Lợi ích: chỗ khác chỉ cần biết **IPlcConnection** là đủ, không quan tâm đó là OPC UA hay Siemens hay Modbus. Đây là chìa khóa để **PlcManager** quản lý nhiều loại PLC bằng cùng một kiểu (xem Chương 4 và 8).
- **abstract class** giống interface nhưng có thể chứa sẵn một phần code dùng chung. **ObservableObject** trong project là abstract class (xem Chương 3).

A.6. Generics (kiểu tổng quát)

```
List<string> names; // List chứa string
List<PlcDevice> devices; // List chứa PlcDevice
ObservableCollection<TagItem> tags; // Collection chứa TagItem
Task<bool> result; // Task trả về bool
ConcurrentDictionary<string, TagValue> cache; // Dictionary: key string, value TagValue
```

<T> là "chỗ trống điền kiểu vào sau". Một **List<T>** viết một lần, dùng cho mọi kiểu. Project dùng generics dày đặc: collections, Task, RelayCommand<T>...

A.7. Nullable — dấu **?**, **!**, **??**, **?.**

Đây là thứ C# hiện đại bắt buộc phải hiểu, nếu không sẽ đọc code không nổi.

```
string name; // KHÔNG được null (non-nullable reference type)
string? error; // CÓ THỂ null (dấu ? sau kiểu)

int? lockMinutes; // int bình thường không null được; int? thì null được

error?.Length // ?. : nếu error null thì trả null luôn, không crash (null-conditional)
error ?? "none" // ?? : nếu error null thì dùng "none" (null-coalescing)
error!.Length // ! : "tôi chắc chắn không null, compiler đừng cảnh báo" (null-forgiving)
```

Ví dụ thật từ project: `LockDurationMinutes` là `int?` với quy ước **null = mặc định, 0 = không giới hạn** — nhờ nullable mới phân biệt được "chưa đặt" với "đặt bằng 0".

A.8. Delegate, Action, Func, và event

Phần này khó với người mới nhưng project dùng rất nhiều (mọi command, mọi event).

```
Action doSomething;           // con trỏ tới hàm KHÔNG tham số, KHÔNG trả về
Action<string> log;           // hàm nhận 1 string, không trả về
Func<bool> canRun;           // hàm không tham số, trả về bool
Func<int, int, int> add;      // hàm nhận 2 int, trả về int

// event: cơ chế "ai quan tâm thì đăng ký, khi có chuyện tôi gọi tất cả"
public event EventHandler<PlcConnectionStateChangedEventArgs>? StateChanged;

StateChanged?.Invoke(this, args); // phát sự kiện; ?. để khỏi crash nếu chưa ai đăng ký
```

Hiểu nôm na: **event** giống như "danh sách số điện thoại". Ai muốn được báo tin thì **+=** (đăng ký), khi có chuyện thì class gọi **Invoke** để "gọi điện" cho tất cả. `PlcManager` subscribe event của từng `PlcConnection` rồi re-fire lên cho ViewModel — đó là **event aggregation** (Chương 8).

```
connection.StateChanged += OnStateChanged; // đăng ký
connection.StateChanged -= OnStateChanged; // hủy đăng ký (QUAN TRỌNG: quên -= sẽ rò rỉ bộ nhớ)
```

A.9. Lambda expression

```
() => DoWork()           // hàm không tham số
x => x * 2                // nhận x, trả x*2
(a, b) => a + b           // nhận a,b trả tổng
u => u.Username == "admin" // dùng trong LINQ để lọc
```

=> đọc là "thì làm" / "ánh xạ tới". Lambda là cách viết hàm ngắn gọn ngay tại chỗ.

A.10. LINQ — truy vấn collection

```
users.FirstOrDefault(u => u.Username == name) // phần tử đầu thỏa điều kiện, hoặc null
users.Where(u => u.IsActive)                  // lọc ra các phần tử thỏa điều kiện
users.Any(u => u.Role == UserRole.Admin)      // có ít nhất 1 admin không? -> bool
users.Select(p => p.EndpointUrl)               // biến List<Plc> thành List<string url>
devices.ToList()                              // chuyển sang List để giữ bản sao
```

LINQ giúp xử lý danh sách bằng câu ngắn gọn thay vì viết vòng **for**. Project dùng LINQ ở khắp `UserService`, `PlcManager`, `ViewModels`.

A.11. async/await, Task, CancellationToken

Quan trọng đến mức có hẳn Phụ lục F riêng. Tóm tắt nhanh:

```
public async Task<bool> ConnectAsync(CancellationToken ct = default)
{
    await Task.Delay(100);    // "chờ" mà không khóa luồng
    return true;
}
```

- `async` đánh dấu hàm có thể "tạm dừng và quay lại".
- `await` = "chờ việc này xong rồi đi tiếp, nhưng đừng đóng băng giao diện".
- `Task` = lời hứa "sẽ có kết quả sau". `Task<bool>` = sẽ có một bool sau.
- `CancellationToken` = "cái nút hủy" để ngừng việc dài giữa chừng.

A.12. using / IDisposable

```
using (var stream = new FileStream(...))    // tự động gọi Dispose() khi ra khỏi khối
{
    // dùng stream
}    // <- stream.Dispose() được gọi tự động ở đây

public class OperatorLockService : IDisposable    // class nắm tài nguyên cần dọn dẹp
{
    public void Dispose() { /* đóng file watcher, timer... */ }
}
```

`IDisposable` + `Dispose()` là cách .NET "dọn rác thủ công" cho thứ GC không tự lo được: file, kết nối mạng, timer, OPC UA session. Project implement `IDisposable` ở `PlcConnection`, `OperatorLockService`, `ApiHostService` ...

A.13. Attribute

```
[JsonIgnore]                // bảo Newtonsoft.Json ĐỪNG lưu property này ra file
public TagQuality Quality { get; set; }

public void OnChanged([CallerMemberName] string? name = null) { }    // tự điền tên property gọi nó
```

Attribute là "ghi chú có ý nghĩa" gắn lên code, được thư viện/khung đọc và xử lý.

`[JsonIgnore]` dùng để **không serialize các giá trị runtime** (Value, Quality, Timestamp) — chỉ lưu cấu hình, không lưu dữ liệu tạm.

Phụ lục B: WPF và XAML nền tảng

B.1. XAML là gì và quan hệ với code-behind

WPF tách giao diện làm 2 file song sinh:

```
MainWindow.xaml      <- mô tả GIAO DIỆN bằng XML (nút, ô, layout)
MainWindow.xaml.cs    <- "code-behind": code C# xử lý sự kiện của cửa sổ đó
```

XAML là XML mô tả cây giao diện. Ví dụ:

```
<Button Content="Lưu" Click="Save_Click" Width="80"/>
```

tương đương C#:

```
var btn = new Button { Content = "Lưu", Width = 80 };
btn.Click += Save_Click;
```

XAML chỉ là cách viết khai báo (declarative) cho cùng việc đó, gọn và dễ nhìn hơn.

B.2. Layout cơ bản

- **Grid**: chia ô theo hàng/cột (giống bảng Excel). Dùng nhiều nhất. `xml <Grid>`
`<Grid.RowDefinitions> <RowDefinition Height="Auto"/> <!-- cao vừa đủ nội dung -->`
`<RowDefinition Height="*" /> <!-- chiếm hết phần còn lại --> </Grid.RowDefinitions>`
`<TextBlock Grid.Row="0" Text="Tiêu đề" /> <ListBox Grid.Row="1" /> </Grid>`
- **StackPanel**: xếp con theo chiều dọc (`Vertical`) hoặc ngang (`Horizontal`).
- **Border**: khung bo góc, đổ bóng — dùng để tạo "thẻ" (card) đẹp như trong các dialog.

B.3. Data Binding — TRÁI TIM của WPF

Đây là khái niệm phải hiểu nhất. Binding = "buộc" một thuộc tính UI vào một property trong code, để **tự động đồng bộ**.

```
<TextBlock Text="{Binding PlcName}" />
```

Nghĩa là: "Text của ô này = giá trị property `PlcName` trong DataContext". Khi `PlcName` đổi, ô tự cập nhật — **không cần viết code gán tay**.

Điều kiện để binding tự cập nhật: property phải báo "tôi vừa đổi" qua

`INotifyPropertyChanged` (xem B.5).

B.4. DataContext

`DataContext` là "nguồn dữ liệu" mà binding tìm tới. Trong project:

```
mainWindow.DataContext = _mainViewModel; // App.xaml.cs
```

Sau dòng này, mọi `{Binding XXX}` trong `MainWindow.xaml` sẽ tìm property `XXX` trong `MainViewModel`. Đây là chỗ **View gặp ViewModel**.

B.5. INotifyPropertyChanged và ObservableObject

```
public abstract class ObservableObject : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;

    protected void OnPropertyChanged([CallerMemberName] string? name = null)
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));

    protected bool SetProperty<T>(ref T field, T value, [CallerMemberName] string? name = null)
    {
        if (Equals(field, value)) return false; // không đổi thì thôi
        field = value;
        OnPropertyChanged(name); // báo UI cập nhật
        return true;
    }
}
```

Cơ chế: khi property đổi → gọi `OnPropertyChanged` → phát event `PropertyChanged` → WPF nghe được → cập nhật mọi ô đang bind tới property đó. **Không có cái này, binding chỉ hiển thị giá trị ban đầu rồi đứng yên.**

`[CallerMemberName]` tự điền tên property gọi nó, nên không phải gõ tay tên (tránh gõ sai → bug âm thầm).

B.6. ObservableCollection

```
public ObservableCollection<PlcDevice> PlcDevices { get; } = new();
```

Khác `List<T>` ở chỗ: khi **thêm/xóa** phần tử, nó tự báo cho UI → `ListBox/DataGrid` tự thêm/xóa dòng. `List<T>` thì không, UI sẽ không biết. Vì vậy mọi danh sách hiển thị trên UI đều dùng `ObservableCollection`.

B.7. Command và ICommand

Thay vì viết `Button.Click` trong code-behind, MVVM bind nút vào một **Command** trong ViewModel:

```
<Button Content="Thêm PLC" Command="{Binding AddPlcCommand}"/>
```

```
public ICommand AddPlcCommand { get; }
// trong constructor:
AddPlcCommand = new RelayCommand(_ => AddPlc(), _ => CanAddPlc());
```

RelayCommand (Chương 5) gói một hàm thường thành **ICommand**. **CanExecute** quyết định nút có **sáng/mờ** hay không. Đây là cách MVVM tách "việc nhấn nút" ra khỏi code giao diện.

B.8. Converter

Khi dữ liệu và hiển thị không cùng kiểu, dùng converter để "phiên dịch":

```
<Border Visibility="{Binding IsConnected, Converter={StaticResource BoolToVis}}"/>
```

BooleanToVisibilityConverter đổi **true/false** → **Visible/Collapsed**. Project có nhiều converter tự viết để đổi trạng thái → màu, → icon, → text.

B.9. Styles, Resources, StaticResource

```
<Window.Resources>
    <Style x:Key="PrimaryButton" TargetType="Button"> ... </Style>
</Window.Resources>

<Button Style="{StaticResource PrimaryButton}"/>
```

- **Resource**: thứ khai báo một lần, tái sử dụng nhiều nơi (style, màu, brush, converter).
- **StaticResource**: tra cứu **một lần lúc tải**. **DynamicResource**: tra cứu lại khi đổi (cho theme động).
- Project gom toàn bộ màu thương hiệu (navy **#1B2C7E**, vàng **#F5C61C**), font, style nút vào file resource dùng chung → giao diện đồng nhất.

B.10. Dispatcher và UI thread — nguồn gốc nhiều bug

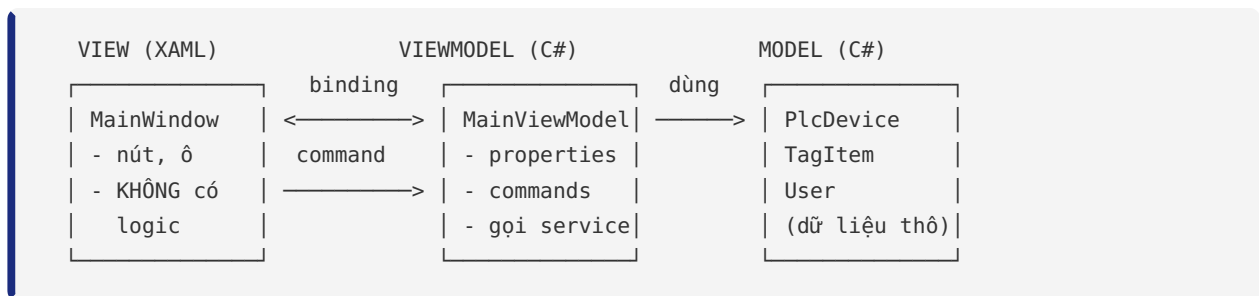
Quy tắc sắt: chỉ UI thread mới được chạm vào UI. Nhưng dữ liệu PLC về từ **background thread** (luồng nền của OPC UA). Nếu background thread đụng thẳng vào UI → app **crash**.

```
Application.Current.Dispatcher.InvokeAsync(() =>
{
    LogEntries.Add(entry); // an toàn vì chạy trên UI thread
});
```


`Dispatcher` là "hàng đợi việc của UI thread". `InvokeAsync` = "nhờ UI thread làm hộ việc này". `ObservableObject.OnPropertyChanged` và `UiLogSink` đều tự động marshal về UI thread qua Dispatcher (Chương 3, 7).

Phụ lục C: MVVM - hiểu thật kỹ

MVVM (Model - View - ViewModel) là kiến trúc xương sống của toàn bộ phần giao diện.



- **Model:** dữ liệu thuần (PlcDevice, TagItem, User). Không biết gì về giao diện.
- **View:** giao diện (XAML). **Không chứa logic nghiệp vụ**, chỉ hiển thị và bắt thao tác.
- **ViewModel:** cầu nối. Giữ trạng thái để View bind vào (properties), giữ hành động để View gọi (commands), và gọi xuống Service/Manager để làm việc thật.

Vì sao tách như vậy? 1. **Test được:** ViewModel là C# thuần, test không cần mở cửa sổ. 2. **Đổi giao diện không đụng logic:** vẽ lại View mà ViewModel giữ nguyên. 3. **Code-behind mỏng:** file `.xaml.cs` gần như trống, mọi thứ nằm ở ViewModel.

Luồng một thao tác điển hình (nhấn "Thêm PLC"): 1. User nhấn nút →

`Command="{Binding AddPlcCommand}"` kích hoạt. 2. `AddPlcCommand` gọi method `AddPlc()` trong ViewModel. 3. ViewModel gọi `PlcManager.AddPlcAsync(...)` (Service layer). 4. Manager tạo `PlcConnection`, kết nối, phát event. 5. ViewModel nghe event, cập nhật `ObservableCollection` và property trạng thái. 6. Nhờ `INotifyPropertyChanged`, UI tự cập nhật. **Không một dòng code nào "vẽ" lại UI thủ công.**

Phụ lục D: OPC UA - khái niệm cốt lõi

Để hiểu `PlcConnection.cs` bạn phải nắm các khái niệm OPC UA sau.

D.1. OPC UA là gì

OPC UA (Unified Architecture) là **chuẩn giao tiếp công nghiệp** cho phép máy tính nói chuyện với PLC/thiết bị của nhiều hãng theo cùng một ngôn ngữ. Mô hình **Client-Server**: PLC (hoặc gateway) chạy **Server**; ứng dụng của chúng ta là **Client**.

D.2. Address Space, Node, và NodeId

Server phơi bày dữ liệu dưới dạng **cây node** (Address Space):

```
Root
├─ Objects    (i=85)  <- nơi chứa dữ liệu thật
├─ Types      (i=86)
└─ Views      (i=87)
```

- **Node**: một điểm trên cây (một biến, một thư mục, một kiểu...).
- **NodeId**: "địa chỉ" duy nhất của node. Có nhiều dạng: `i=85` (numeric), `ns=2;s=Temperature` (string trong namespace 2)...

BUG KINH ĐIỂN (Chương 14):

```
new NodeId("i=85")    // SAI: tạo NodeId kiểu STRING với nội dung "i=85"
NodeId.Parse("i=85")  // ĐÚNG: phân tích ra NodeId kiểu NUMERIC = 85
```

Server hiểu `Parse` nhưng từ chối cái kia với lỗi `BadNodeIdUnknown`. Phải hiểu `NodeId` mới thấy được lỗi này.

D.3. Browse

Browse = "duyet cây": hỏi server "node này có những con nào?". `BrowseServerWindow` dùng browse để hiển thị cây tag, lazy-load từng nhánh khi người dùng mở rộng (Chương 13).

D.4. Session, Subscription, MonitoredItem

```
Session                <- 1 phiên kết nối Client-Server (như 1 lần đăng nhập)
├─ Subscription        <- 1 nhóm theo dõi, có chu kỳ publish riêng
│   └─ MonitoredItem    <- 1 tag được theo dõi
└─ MonitoredItem
```

- **Session**: phiên làm việc. Mất mạng → session đứt → cần reconnect.
- **Subscription**: đăng ký nhận thay đổi. Thay vì hỏi liên tục (polling), bạn nói "có gì đổi thì báo tôi".
- **MonitoredItem**: từng tag cụ thể trong subscription. Khi giá trị đổi vượt ngưỡng, server **đẩy** về client → event `TagValueChanged`.

Mô hình **đẩy** (push) này hiệu quả hơn hỏi liên tục, đặc biệt với hàng trăm tag.

D.5. Reconnect 2 lớp

Project chống mất kết nối bằng 2 lớp (Chương 8): 1. **SessionReconnectHandler** (SDK lo sẵn): tự nối lại khi rớt mạng thoáng qua. 2. **AutoReconnectLoopAsync** (tự viết): khi

rất lâu, vòng lặp thử lại với **exponential backoff** (chờ 2s, 4s, 8s, 16s... tăng dần) để không spam server.

Sau khi nối lại có **settling period 2000ms**: bỏ qua các event KeepAlive cũ còn "lởn vờn" để tránh hiểu nhầm trạng thái.

D.6. Security

OPC UA có **SecurityPolicy** (thuật toán mã hóa) và **SecurityMode** (None / Sign / SignAndEncrypt) cùng **certificate** (chứng chỉ để hai bên tin nhau). `SelectBestEndpoint` chọn endpoint an toàn nhất mà cả hai cùng hỗ trợ (Chương 8).

Phụ lục E: Dependency Injection và vòng đời object

E.1. Vấn đề DI giải quyết

Không có DI, class tự tạo thứ nó cần:

```
public class MainViewModel
{
    private PlcManager _manager = new PlcManager(new ConfigurationService(), ...); // ràng buộc cứng
}
```

Cách này khó test, khó thay thế, khó tái dùng. **Dependency Injection** đảo ngược: class chỉ **khai báo** thứ nó cần qua constructor, có người khác **đưa vào**.

```
public class MainViewModel
{
    private readonly IPlcManager _manager;
    public MainViewModel(IPlcManager manager) // "tôi cần 1 IPlcManager, ai đó đưa cho tôi"
    {
        _manager = manager;
    }
}
```

E.2. Container và đăng ký service

`App.xaml.cs` dựng một **container** (kho) khai báo "khi cần kiểu X, hãy đưa object Y":

```

var services = new ServiceCollection();
services.AddSingleton<IPlcManager, PlcManager>(); // 1 instance dùng chung cả app
services.AddSingleton<UserService>();
services.AddSingleton<MainViewModel>();
services.AddTransient<MainWindow>(); // mỗi lần xin là 1 instance mới
_serviceProvider = services.BuildServiceProvider();

var vm = _serviceProvider.GetRequiredService<MainViewModel>(); // container tự lắp ráp mọi dependency

```

Container tự nhìn constructor, thấy **MainViewModel** cần **IPlcManager**, nó tự tạo/đưa vào — **đệ quy** cho cả chuỗi phụ thuộc.

E.3. Lifetime — vòng đời

Lifetime	Ý nghĩa	Dùng khi
Singleton	1 instance cho cả app	Service giữ trạng thái dùng chung (PlcManager, UserService)
Transient	Mỗi lần xin là 1 cái mới	Cửa sổ, dialog
Scoped	1 instance/phạm vi (mỗi request HTTP)	Controller trong API

Hiểu lifetime giúp tránh bug "tại sao dữ liệu bị chia sẻ ngoài ý muốn" hoặc "tại sao mỗi lần lại mất state".

Phụ lục F: async/await và lập trình bất đồng bộ

F.1. Vì sao cần async

Nếu kết nối PLC (mất vài giây) chạy trên UI thread theo kiểu **đồng bộ**, giao diện **đơ cứng** suốt thời gian đó. **async/await** cho phép "chờ" mà UI vẫn mượt.

```

public async Task<bool> ConnectAsync(Cancellation token ct = default)
{
    UpdateState(Connecting);
    var session = await Session.CreateAsync(...); // chờ ~vài giây, UI KHÔNG đơ
    UpdateState(Connected);
    return true;
}

```

F.2. Hiểu Task, async, await

- Task** = "lời hứa có kết quả sau". **Task<bool>** = sẽ có bool. **Task** không kèm kiểu = chỉ "sẽ xong, không trả gì".

- `async` = đánh dấu hàm chứa `await`, cho phép tạm dừng-quay lại.
- `await` = "chờ Task này xong rồi đi tiếp; trong lúc chờ, trả luồng lại cho người khác dùng".

F.3. Các "bẫy" async trong project

1. `async void` chỉ dùng cho event handler. Lý do: `async void` không await được, lỗi không bắt được. Nhưng `ICommand.Execute` bắt buộc trả `void`, nên `AsyncRelayCommand.Execute` phải là `async void` — và phải bọc `try/finally` để không nuốt lỗi (Chương 5).

2. Re-entry guard. `AsyncRelayCommand` dùng cờ `_isExecuting` để chặn double-click khi tác vụ async đang chạy.

3. Fire-and-forget có chủ đích.

```
_ = InitializeViewModelAsync(_mainViewModel); // App.xaml.cs: cố ý KHÔNG await
```

Dấu `_ =` nghĩa "tôi biết mình không await". Mục đích: cho `mainWindow.Show()` chạy trước để UI hiện ngay, rồi mới khởi tạo nền (Chương 12).

4. CancellationToken. Truyền `ct` xuống các tầng để có thể hủy tác vụ dài (đóng app giữa lúc đang kết nối).

F.4. ConcurrentDictionary và thread-safety

Khi nhiều luồng cùng đọc/ghi một collection, `Dictionary` thường sẽ hỏng dữ liệu. `ConcurrentDictionary` an toàn cho đa luồng — dùng cho `DataCacheService` (REST API thread ghi/đọc song song) và quản lý connections trong `PlcManager`.

Ngoài ra project còn dùng `lock (_lock) { ... }` (ví dụ `UserService`) để đảm bảo một khối code chỉ một luồng vào tại một thời điểm.

Phụ lục G: Cấu trúc dự án và file .csproj

G.1. Sơ đồ thư mục

```
OPCUACommDriver/
├─ App.xaml / App.xaml.cs      <- điểm khởi động, DI, logging
├─ OPCUACommDriver.csproj      <- file dự án: target framework, NuGet packages
├─ Enums/                      <- các kiểu liệt kê
├─ Models/                     <- lớp dữ liệu (PlcDevice, TagItem, User, ObservableObject)
├─ Interfaces/                 <- IPlcConnection, IPlcManager...
├─ Helpers/                    <- RelayCommand, Converters
├─ Services/
│   ├─ Auth/                   <- UserService, AuthService, OperatorLockService
│   ├─ OpcUa/                  <- PlcConnection, PlcManager
│   ├─ Protocols/              <- Siemens, Mitsubishi, Modbus
│   └─ ConfigurationService.cs
├─ DataCacheService.cs
├─ UiLogSink.cs
├─ Api/                         <- ApiHostService, Controllers, Hubs (ASP.NET Core nhúng)
├─ ViewModels/                 <- MainViewModel, BrowseServerViewModel
├─ Views/                      <- MainWindow, LoginWindow, các Dialog (.xaml + .xaml.cs)
├─ Configurations/             <- file JSON lưu cấu hình & lock_state
└─ Logs/                       <- file log theo ngày
```

G.2. File .csproj — "bản kê khai" của dự án

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>      <!-- ứng dụng cửa sổ Windows -->
    <TargetFramework>net8.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>                 <!-- bật WPF -->
    <Nullable>enable</Nullable>           <!-- bật nullable reference types (dấu ?) -->
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="OPCFoundation.NetStandard.Opc.Ua" Version="1.5.374.126"/>
    <PackageReference Include="Serilog" .../>
    <PackageReference Include="BCrypt.Net-Next" .../>
    <PackageReference Include="Newtonsoft.Json" .../>
    <PackageReference Include="Microsoft.Extensions.DependencyInjection" .../>
    <!-- ASP.NET Core, JWT, SignalR... -->
  </ItemGroup>
</Project>
```

<PackageReference> là khai báo NuGet package. Khi build, NuGet tải các thư viện này về.

G.3. Các package chính và vai trò

Package	Vai trò trong dự án
OPCFoundation.NetStandard.Opc.Ua	Bộ SDK OPC UA: Session, Subscription, Browse...
Serilog (+ sinks)	Ghi log ra console, file, và UI
BCrypt.Net-Next	Băm mật khẩu (workFactor=11)
Newtonsoft.Json	Đọc/ghi file cấu hình JSON
Microsoft.Extensions.DependencyInjection	Container DI
Microsoft.AspNetCore.*	API REST + SignalR nhúng trong WPF
JwtBearer	Xác thực JWT cho API

Phụ lục H: Dựng lại dự án từ con số 0 (checklist)

Đây là thứ tự thực tế để xây lại app, theo đúng chiều phụ thuộc (thứ ít phụ thuộc làm trước). Khớp với Chương 15.

1. **Tạo project WPF .NET 8.** `dotnet new wpf`, sửa `.csproj` thành `net8.0-windows`, bật `Nullable`. Cài các NuGet ở Phụ lục G.3.
2. **Enums.** Tạo toàn bộ enum (UserRole, PlcConnectionState, TagQuality, ProtocolType...). Không phụ thuộc gì → làm đầu tiên.
3. **Models nền tảng.** `ObservableObject` (base cho binding), rồi `User`, `PlcDevice`, `TagItem`. Nhớ `[JsonIgnore]` cho giá trị runtime.
4. **Interfaces.** `IPlcConnection`, `IPlcManager` — định nghĩa hợp đồng trước khi viết phần thân.
5. **Helpers.** `RelayCommand`, `AsyncRelayCommand`, `RelayCommand<T>`, các Converter.
6. **Services hạ tầng.** `ConfigurationService` (JSON), `UiLogSink` (Serilog→UI), `DataCacheService`.
7. **Services/Auth.** `UserService` (BCrypt, tự tạo admin/admin123), `AuthService` (JWT), `OperatorLockService` (file lock + watcher).
8. **OPC UA core.** `PlcConnection` (chú ý `NodeId.Parse`, reconnect 2 lớp, Logger là property), rồi `PlcManager` (ConcurrentDictionary, event aggregation).
9. **Protocols khác** (tùy chọn): Siemens, Mitsubishi, Modbus — implement cùng `IPlcConnection`.
10. **API layer.** `ApiHostService` nhúng ASP.NET Core, các Controller, SignalR Hub.
11. **ViewModels.** `MainViewModel` (wire mọi command + event), `BrowseServerViewModel`.

12. **Views.** `LoginWindow` trước, rồi `MainWindow` , `BrowseServerWindow` , các Dialog. Đặt resource (màu, style) dùng chung.

13. **App.xaml.cs.** Ghép tất cả: DI, logging 2 pha, login→main flow, fire-and-forget init, khởi động API. **Làm cuối cùng** vì nó phụ thuộc mọi thứ.

Mẹo: build và chạy được sau mỗi bước lớn (sau 6, sau 8, sau 12). Đừng viết hết rồi mới chạy — sẽ ngập trong lỗi.

Phụ lục I: Bảng thuật ngữ (Glossary)

Thuật ngữ	Nghĩa ngắn gọn
WPF	Windows Presentation Foundation — khung làm app desktop Windows
XAML	Ngôn ngữ XML mô tả giao diện WPF
Code-behind	File <code>.xaml.cs</code> đi kèm file XAML
MVVM	Model-View-ViewModel: kiến trúc tách giao diện và logic
Binding	"Buộc" thuộc tính UI vào property code để tự đồng bộ
DataContext	Nguồn dữ liệu mà binding tra cứu
INotifyPropertyChanged	Cơ chế báo "property đã đổi" cho UI cập nhật
ObservableCollection	Danh sách tự báo UI khi thêm/xóa phần tử
ICommand / RelayCommand	Đối tượng đại diện cho một hành động nút bấm trong MVVM
Converter	Bộ "phiên dịch" giá trị giữa code và UI
Dispatcher	Hàng đợi việc của UI thread
UI thread	Luồng duy nhất được phép chạm vào giao diện
DI / Dependency Injection	Đưa dependency vào qua constructor thay vì tự tạo
Container / ServiceProvider	Kho đăng ký và cấp phát object cho DI
Singleton / Transient / Scoped	Các vòng đời object trong DI
async / await	Cú pháp lập trình bất đồng bộ
Task	"Lời hứa" sẽ có kết quả trong tương lai
CancellationToken	"Nút hủy" tác vụ dài
ConcurrentDictionary	Dictionary an toàn cho đa luồng
IDisposable / Dispose	Cơ chế dọn dẹp tài nguyên thủ công
Generic <code><T></code>	Kiểu tổng quát, điền kiểu cụ thể sau
Nullable <code>?</code>	Cho phép giá trị null; đi kèm <code>??</code> , <code>?.</code> , <code>!</code>
Delegate / Action / Func	Con trỏ tới hàm
Event	Cơ chế "đăng ký - nhận thông báo"
Lambda <code>=></code>	Cách viết hàm ngắn gọn tại chỗ
LINQ	Bộ truy vấn collection (Where, Select, FirstOrDefault...)
Attribute <code>[...]</code>	Ghi chú có ý nghĩa gắn lên code

Thuật ngữ	Nghĩa ngắn gọn
OPC UA	Chuẩn giao tiếp công nghiệp Client-Server
NodeId	Địa chỉ duy nhất của một node trên OPC UA server
Browse	Duyệt cây node của server
Session	Phiên kết nối OPC UA
Subscription	Đăng ký nhận thay đổi giá trị
MonitoredItem	Một tag được theo dõi trong subscription
Exponential backoff	Chiến lược thử lại với thời gian chờ tăng dần
JWT	JSON Web Token — vé xác thực cho API
BCrypt	Thuật toán băm mật khẩu chậm có chủ đích
Serilog	Thư viện logging có cấu trúc
PLC	Programmable Logic Controller — bộ điều khiển trong nhà máy
Tag	Một biến/điểm dữ liệu trong PLC
NuGet	Trình quản lý gói thư viện của .NET
.csproj	File mô tả dự án .NET

Lời khuyên cuối: Khi đọc lại một file code bất kỳ, hãy tự hỏi 3 câu: (1) File này thuộc **lớp nào** (Model/Service/ViewModel/View)? (2) Nó **phụ thuộc** vào gì và **ai phụ thuộc** vào nó? (3) Dữ liệu **đi vào và đi ra** bằng đường nào (binding, event, hay gọi method)? Trả lời được 3 câu này là bạn đã "lấy lại gốc".